



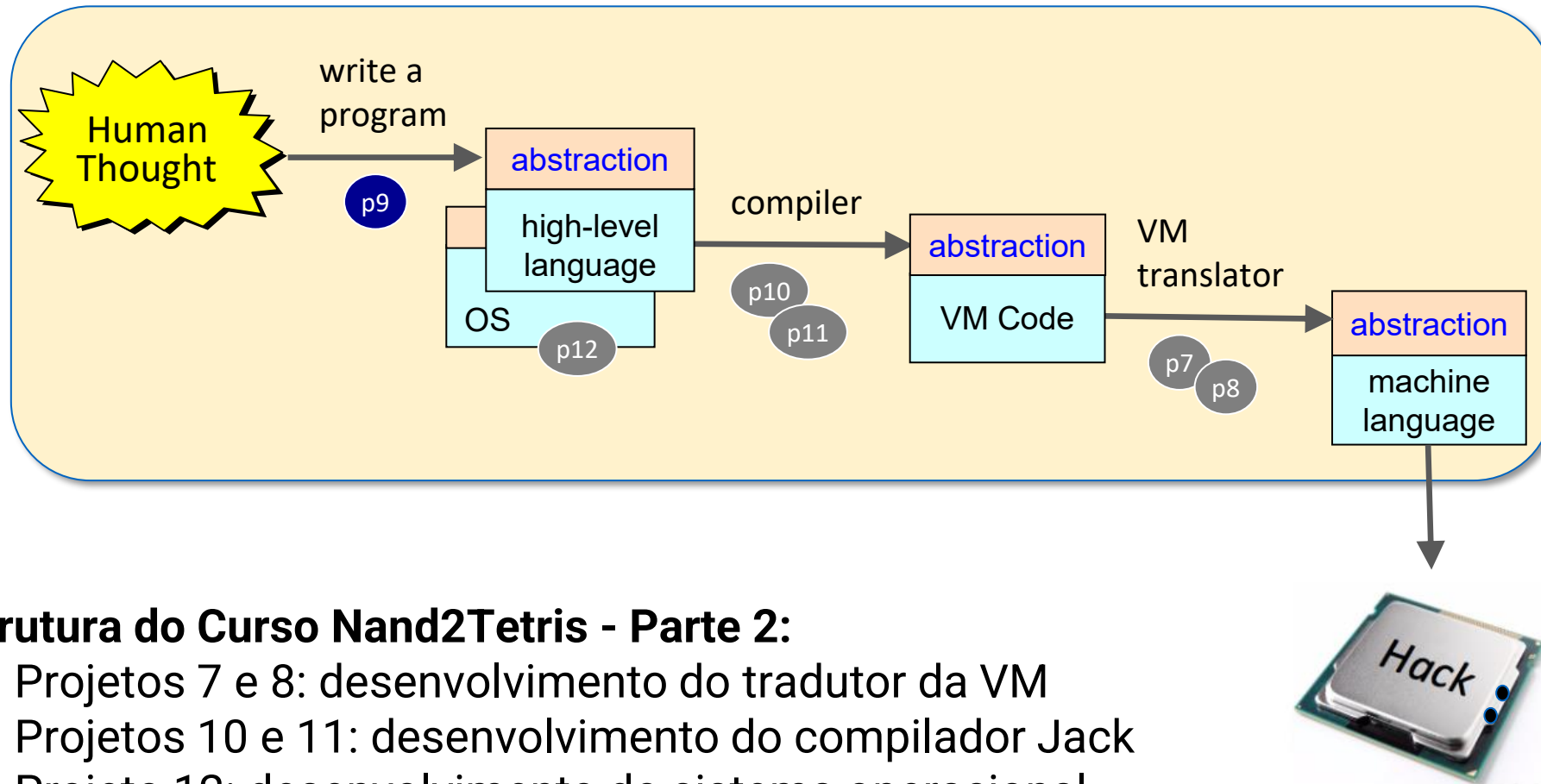
UNIVERSIDADE
FEDERAL RURAL
DE PERNAMBUCO

Capítulo 9 - Linguagem de Alto Nível

Elementos de Sistemas Computacionais

Professor: Lucas Cambuim
lucas.cambuim@ufrpe.br

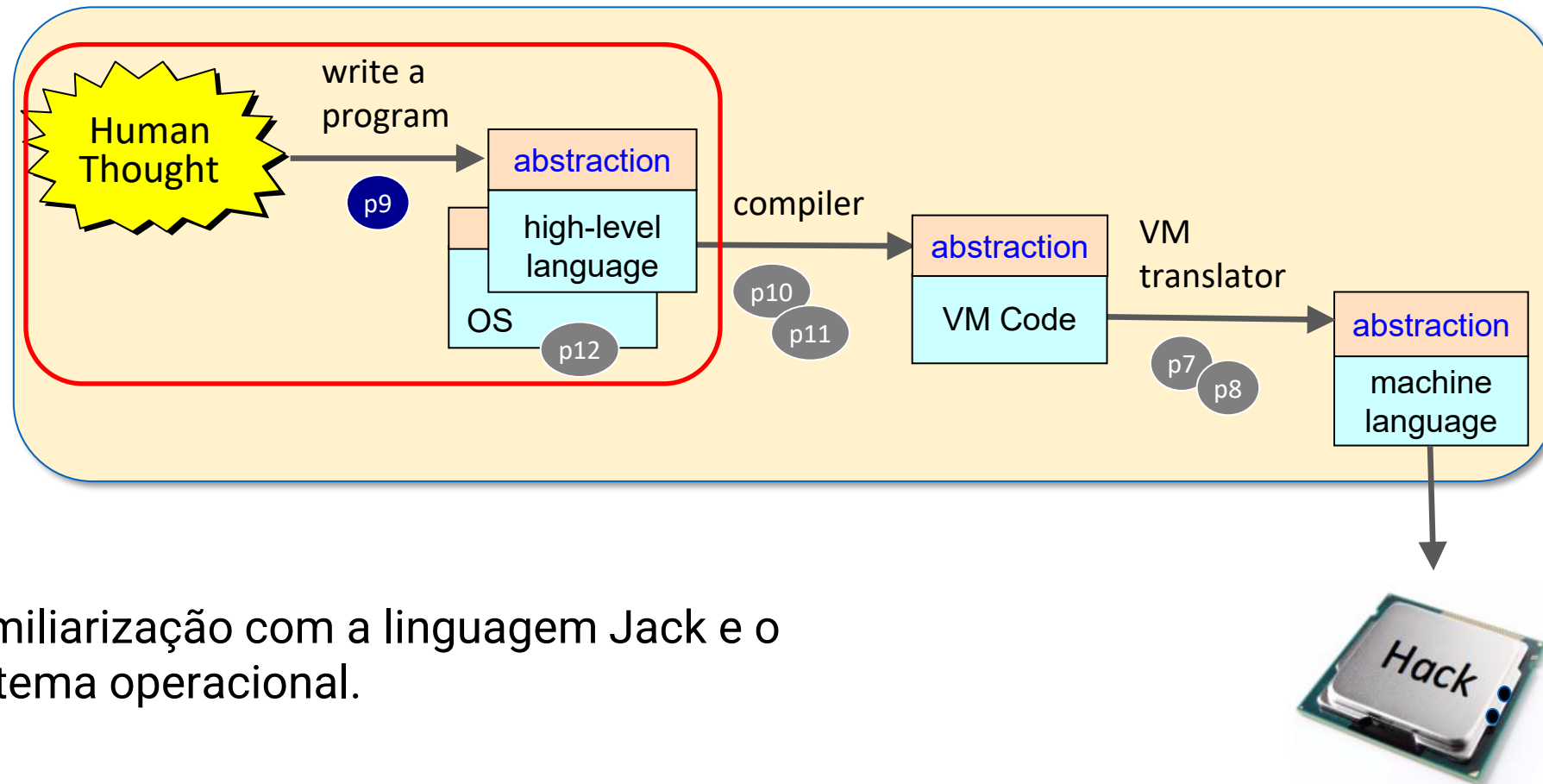
Roadmap



- **Estrutura do Curso Nand2Tetris - Parte 2:**

- Projetos 7 e 8: desenvolvimento do tradutor da VM
- Projetos 10 e 11: desenvolvimento do compilador Jack
- Projeto 12: desenvolvimento do sistema operacional
- **Projeto 9: visão geral da linguagem Jack**

Roadmap



- Familiarização com a linguagem Jack e o sistema operacional.

Jack em poucas palavras

```
/** Demo: working with Point objects. */
class Main {
  function void main() {
    var Point p1, p2, p3;
    let p1 = Point.new(1,2);
    let p2 = Point.new(3,4);
    let p3 = p1.plus(p2);
    do p3.print();
    do Output.println();
    do Output.printInt(p1.distance(p3));
    return;
  }
}
```

```
/** Represents a Point object. */
class Point {
  field int x, y;
  static int pointCount;
  /** Constructs a new point */
  constructor Point new(int ax, int ay) {
    let x = ax;
    let y = ay;
    let pointCount = pointCount + 1;
    return this;
  }
  // More Point methods...
}
```

- Uma linguagem simples, *Java-like*
- Orientada à objetos, sem herança
- Propósito geral
- Permite desenvolver aplicações interativas
- Pode ser aprendida em torno de uma hora



Lições importantes a serem aprendidas

- Jack é uma língua legal...



Lições importantes a serem aprendidas

- ...Mas o objetivo não é aprender Jack.



Lições importantes a serem aprendidas

- O objetivo é **Entender como linguagens de alto nível**:
 - Lidam com tipos primitivos e classes
 - Manipulam strings, arrays e listas
 - Criam, representam e descartam objetos
 - Interação com o sistema operacional host
 - Abstração/Implementação
 - Programação orientada a objetos
 - Projeto de aplicações
 - Otimização



Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Hello world

```
/** Hello World program. */  
class Main {  
    function void main() {  
        /* Prints some text using the standard library. */  
        do Output.printString("Hello world!");  
        do Output.println();      // New line  
        return;  
    }  
}
```



Hello world

```
/** Hello World program. */  
class Main {  
    function void main() {  
        /* Prints some text using the standard library. */  
        do Output.printString("Hello world!");  
        do Output.println();      // New line  
        return;  
    }  
}
```



- Onde o programa irá iniciar a execução.

Hello world

```
/** Hello World program. */  
class Main {  
    function void main() {  
        /* Prints some text using the standard library. */  
do Output.println("Hello world!");  
        do Output.println();      // New line  
        return;  
    }  
}
```



- do irá executar algum comando

Hello world

```
/** Hello World program. */  
class Main {  
    function void main() {  
        /* Prints some text using the standard library. */  
        do Output printString("Hello world!");  
        do Output.println();      // New line  
        return;  
    }  
}
```



- Uso de biblioteca da API do SO

Hello world

```
/** Hello World program. */  
class Main {  
    function void main() {  
        /* Prints some text using the standard library. */  
        do Output println("Hello world!");  
        do Output.println();    // New line  
        return;  
    }  
}
```



- Imprime na tela o Hello world

Hello world

```
/** Hello World program. */  
class Main {  
    function void main() {  
        /* Prints some text using the standard library. */  
        do Output.printString("Hello world!");  
        do Output.println();    // New line  
        return;  
    }  
}
```



- Quebra de linha

Hello world

```
/** Hello World program. */  
class Main {  
    function void main() {  
        /* Prints some text using the standard library. */  
        do Output.printString("Hello world!");  
        do Output.println();      // New line  
        return;  
    }  
}
```



- Comando obrigatório em Jack para indicar final de programa

Estruturas da linguagem

```
/** Hello World program. */  
class Main {  
    function void main() {  
        /* Prints some text using the standard library. */  
        do Output.printString("Hello world!");  
        do Output.println();      // New line  
        return;  
    }  
}
```

- Comentários:

/** API block comment */

/* block comment */

// in-line comment

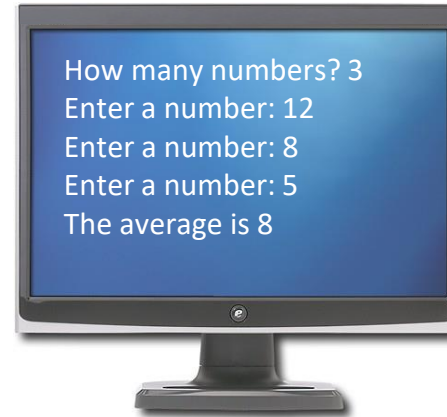
- Espaços em branco:
(ignorado)



Programação procedural

código Jack

```
// Inputs some numbers and computes their average
class Main {
  function void main() {
    var Array a;
    var int length;
    var int i, sum;
    let length = Keyboard.readInt("How many numbers? ");
    let a = Array.new(length); // constructs the array
    let i = 0;
    while (i < length) {
      let a[i] = Keyboard.readInt("Enter a number: ");
      let sum = sum + a[i];
      let i = i + 1;
    }
    do Output.printString("The average is ");
    do Output.printInt(sum / length);
    return;
  }
}
```



Estruturas da linguagem

código Jack

```
// Inputs some numbers and computes their average
```

```
class Main {  
    function void main() {  
        var Array a;  
        var int length;  
        var int i, sum;  
        let length = Keyboard.readInt("How many numbers? ");  
        let a = Array.new(length); // constructs the array  
        let i = 0;  
        while (i < length) {  
            let a[i] = Keyboard.readInt("Enter a number: ");  
            let sum = sum + a[i];  
            let i = i + 1;  
        }  
        do Output.printString("The average is ");  
        do Output.printInt(sum / length);  
        return;  
    }  
}
```

- Um programa Jack é uma coleção de uma ou mais classes Jack, onde uma delas necessariamente precisa ser a Main
- A classe Main deve ter ao menos uma função chamada main
- O ponto de entrada de um programa é a função: Main.main



UFRPE

Estruturas da linguagem

código Jack

```
// Inputs some numbers and computes their average
```

```
class Main {  
    function void main() {  
        var Array a;  
        var int length;  
        var int i, sum;  
        let length = Keyboard.readInt("How many numbers? ");  
        let a = Array.new(length); // constructs the array  
        let i = 0;  
        while (i < length) {  
            let a[i] = Keyboard.readInt("Enter a number: ");  
            let sum = sum + a[i];  
            let i = i + 1;  
        }  
        do Output.printString("The average is ");  
        do Output.printInt(sum / length);  
        return;  
    }  
}
```

- Bons hábitos de código: classes começam com letra **maiúscula** e funções com letra minúscula



UFRPE

Estruturas da linguagem

código Jack

```
// Inputs some numbers and computes their average
class Main {
  function void main() {
    var Array a;
    var int length;
    var int i, sum;
    let length = Keyboard.readInt("How many numbers? ");
    let a = Array.new(length); // constructs the array
    let i = 0;
    while (i < length) {
      let a[i] = Keyboard.readInt("Enter a number: ");
      let sum = sum + a[i];
      let i = i + 1;
    }
    do Output.printString("The average is ");
    do Output.printInt(sum / length);
    return;
  }
}
```

- Tipos de dados Jack:
 - Primitivos:
 - int
 - char
 - boolean
 - Baseados em Classes:
 - Array, String, ...



Estruturas da linguagem

código Jack

```
// Inputs some numbers and computes their average
class Main {
  function void main() {
    var Array a;
    var int length;
    var int i, sum;
    let length = Keyboard.readInt("How many numbers? ");
    let a = Array.new(length); // constructs the array
    let i = 0;

    while (i < length) {
      let a[i] = Keyboard.readInt("Enter a number: ");
      let sum = sum + a[i];
      let i = i + 1;
    }

    do Output.printString("The average is ");
    do Output.printInt(sum / length);
    return;
  }
}
```

- Controle de fluxo:
 - if / if...else
 - while
 - do



Estruturas da linguagem

código Jack

```
// Inputs some numbers and computes their average
class Main {
  function void main() {
    var Array a;
    var int length;
    var int i, sum;
    let length = Keyboard.readInt("How many numbers? ");
    let a = Array.new(length); // constructs the array
    let i = 0;

    while (i < length) {
      let a[i] = Keyboard.readInt("Enter a number: ");
      let sum = sum + a[i];
      let i = i + 1;
    }

    do Output.printString("The average is ");
    do Output.printInt(sum / length);
    return;
  }
}
```

- Controle de fluxo:
 - if / if...else
 - while
 - do

usado para chamar métodos no lado de fora de uma expressão



Estruturas da linguagem

código Jack

```
// Inputs some numbers and computes their average
class Main {
  function void main() {
    var Array a;
    var int length;
    var int i, sum;
    let length = Keyboard.readInt("How many numbers? ");
    let a = Array.new(length); // constructs the array
    let i = 0;
    while (i < length) {
      let a[i] = Keyboard.readInt("Enter a number: ");
      let sum = sum + a[i];
      let i = i + 1;
    }
    do Output.printString("The average is ");
    do Output.printInt(sum / length);
    return;
  }
}
```

- Arrays:
 - O Array é implementado como parte da biblioteca padrão Jack
- Os arrays em Jack não são tipados.



UFRPE

Estruturas da linguagem

código Jack

```
// Inputs some numbers and computes their average
class Main {
  function void main() {
    var Array a;
    var int length;
    var int i, sum;
    let length = Keyboard.readInt("How many numbers? ");
    let a = Array.new(length); // constructs the array
    let i = 0;
    while (i < length) {
      let a[i] = Keyboard.readInt("Enter a number: ");
      let sum = sum + a[i];
      let i = i + 1;
    }
    do Output.printString("The average is ");
    do Output.printInt(sum / length);
    return;
  }
}
```

- Serviços do SO:
 - Keyboard.readInt
 - Output.printString
 - Output.printInt
 - mais...



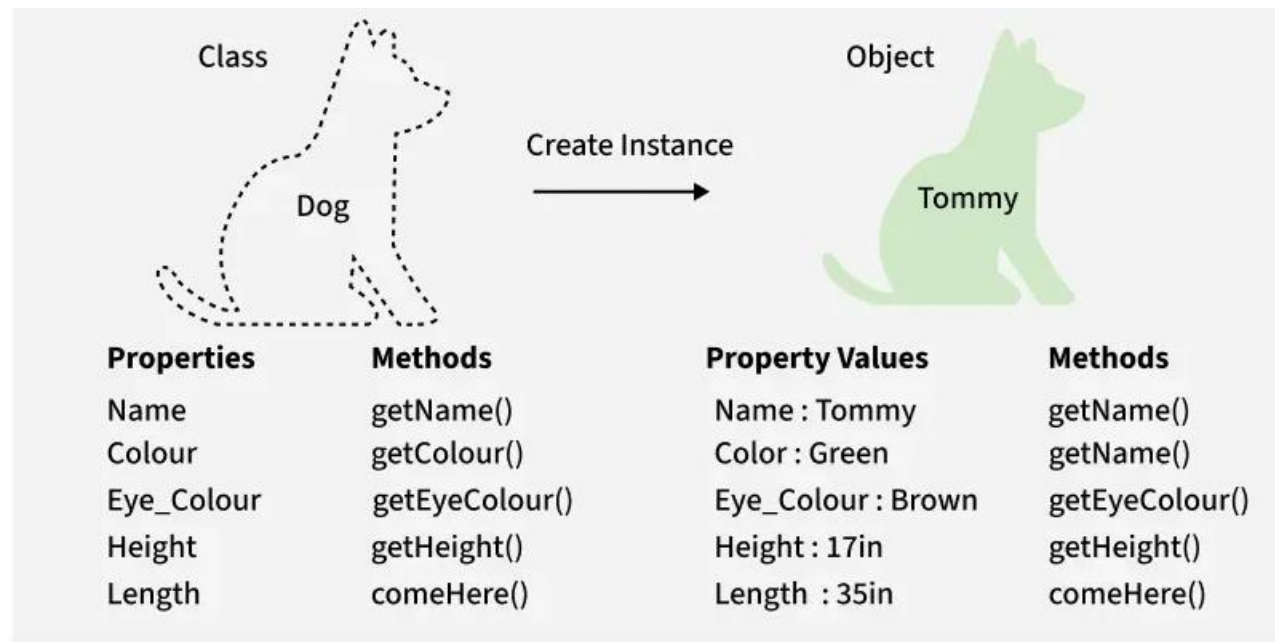
UFRPE

Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Programação OO

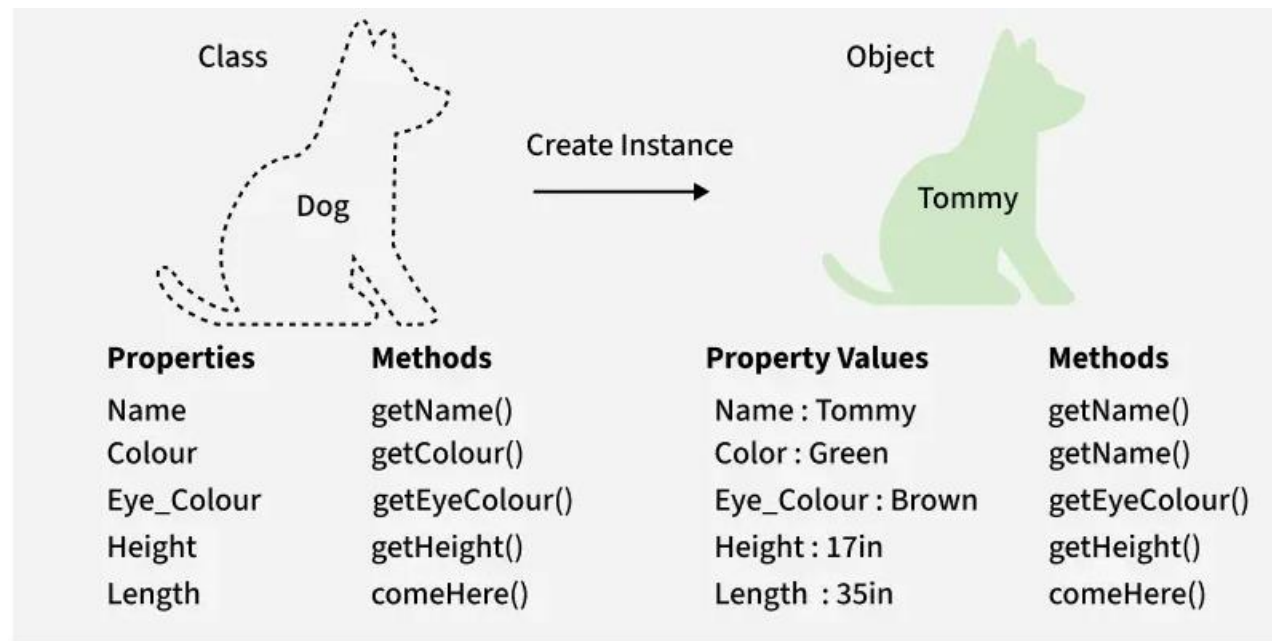
- Uma **classe** é um **modelo** para criar **objetos** com propriedades e comportamentos semelhantes, ou, em outras palavras, podemos dizer que uma classe é um projeto para objetos.



Fonte: <https://www.geeksforgeeks.org/javascript/classes-and-objects-in-javascript/>

Programação OO

- Um **objeto** é uma **instância** de uma classe.
- Por exemplo, o tipo de animal Cachorro é uma classe, enquanto um cachorro específico chamado Tommy é um objeto da classe Cachorro.



Fonte: <https://www.geeksforgeeks.org/javascript/classes-and-objects-in-javascript/>

Programação 00

- Jack não tem o tipo primitivo ponto flutuante
- Desejamos dotar Jack com a capacidade de lidar com números racionais como $2/3$ e $314159/100000$ sem perda de precisão.



Programação OO

- Em linguagens orientadas a objetos, os programadores podem **introduzir novos tipos** criando classes que representam tipos de dados abstratos

Fraction API



```
class Fraction {  
    /** Constructs a (reduced) fraction from the given numerator and denominator */  
    constructor Fraction new(int x, int y)  
  
    /** Accessors. */  
    method int getNumerator()  
    method int getDenominator()  
  
    /** Returns the sum of this fraction and the other one. */  
    method Fraction plus(Fraction other)  
  
    /** Disposes this fraction. */  
    method void dispose()  
  
    /** Prints this fraction in the format x/y. */  
    method void print()  
}
```

Example: fractions

$2/3 + 1/5$

$(2/7 * 3/4) + 1/2$

$8/9 / (5/6 - 2/19)$



UFRPE

Programação OO

- Pode-se desenvolver uma classe Jack independente, projetada para criar e manipular objetos fracionários da forma x/y , onde x e y são inteiros.

Fraction API

```
class Fraction {  
    /** Constructs a (reduced) fraction from the given numerator and denominator */  
    constructor Fraction new(int x, int y)  
  
    /** Accessors. */  
    method int getNumerator()  
    method int getDenominator()  
  
    /** Returns the sum of this fraction and the other one. */  
    method Fraction plus(Fraction other)  
  
    /** Disposes this fraction. */  
    method void dispose()  
  
    /** Prints this fraction in the format x/y. */  
    method void print()  
}
```

Example: fractions

$2/3 + 1/5$

$(2/7 * 3/4) + 1/2$

$8/9 / (5/6 - 2/19)$



UFRPE

Programação OO

- Pode-se desenvolver uma classe Jack independente, projetada para criar e manipular objetos fracionários da forma x/y , onde x e y são inteiros.

Fraction API

```
class Fraction {  
    /** Constructs a (reduced) fraction from the given numerator and denominator */  
    constructor Fraction new(int x, int y)  
  
    /** Accessors. */  
    method int getNumerator()  
    method int getDenominator()  
  
    /** Returns the sum of this fraction and the other one. */  
    method Fraction plus(Fraction other)  
  
    /** Disposes this fraction. */  
    method void dispose()  
  
    /** Prints this fraction in the format x/y. */  
    method void print()  
}
```

Example: fractions

$2/3 + 1/5$

$(2/7 * 3/4) + 1/2$

$8/9 / (5/6 - 2/19)$

Programação OO

- Essa classe pode então fornecer uma **abstração** de fração para qualquer programa Jack que precise representar e manipular números racionais.

Fraction API

```
class Fraction {  
    /** Constructs a (reduced) fraction from the given numerator and denominator */  
    constructor Fraction new(int x, int y)  
  
    /** Accessors. */  
    method int getNumerator()  
    method int getDenominator()  
  
    /** Returns the sum of this fraction and the other one. */  
    method Fraction plus(Fraction other)  
  
    /** Disposes this fraction. */  
    method void dispose()  
  
    /** Prints this fraction in the format x/y. */  
    method void print()  
}
```

Example: fractions

$2/3 + 1/5$

$(2/7 * 3/4) + 1/2$

$8/9 / (5/6 - 2/19)$



UFRPE

Programação OO: utilizando as classes

Fraction API

```
class Fraction {  
    /** Constructs a (reduced) fraction from the given numerator and denominator */  
    constructor Fraction new(int x, int y)  
  
    /** Accessors. */  
    method int getNumerator()  
    method int getDenominator()  
  
    /** Returns the sum of this fraction and the other one. */  
    method Fraction plus(Fraction other)  
  
    /** Disposes this fraction. */  
    method void dispose()  
  
    /** Prints this fraction in the format x/y. */  
    method void print()  
}
```

- Imagina que quiséssemos computar a soma de $2/3$ e $1/5$...

Jack class

```
// Computes the sum of 2/3 and 1/5.  
class Main {  
    function void main() {
```



Programação OO: utilizando as classes

Fraction API

```
class Fraction {  
    /** Constructs a (reduced) fraction from the given numerator and denominator */  
    constructor Fraction new(int x, int y)  
  
    /** Accessors. */  
    method int getNumerator()  
    method int getDenominator()  
  
    /** Returns the sum of this fraction and the other one. */  
    method Fraction plus(Fraction other)  
  
    /** Disposes this fraction. */  
    method void dispose()  
  
    /** Prints this fraction in the format x/y. */  
    method void print()  
}
```



Jack class

```
// Computes the sum of 2/3 and 1/5.  
class Main {  
    function void main() {  
        var Fraction a;  
        let a = Fraction.new(2,3);  
        var Fraction b;  
        let b = Fraction.new(1,5);  
        do a.plus(b);  
    }  
}
```

- **Abstração - Implementação**
 - Usuários não precisam saber nada sobre sua implementação
 - Tudo o que eles precisam é a **interface da classe (API)**

Programação OO: acessando os atributos

classe Fraction

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;

    /** Accessors. */
    method int getNumerator() {
        return numerator;
    }

    method int getDenominator() {
        return denominator;
    }

    // More Fraction methods follow.
} // Fraction class
```

- Para acessar esses parâmetros, em geral não acessamos eles diretamente

Field especifica as propriedades do objeto.

A única maneira de acessar os valores dos campos de fora da classe é por meio de métodos de acesso (get).

Exemplo de classe Jack

```
class Foo {
    ...
    var Fraction x;
    let x = Fraction.new(5,17);
    do Output.printInt(x.numerator); // not allowed
    do Output.printInt(x.getNumerator()); // ok
    ...
}
```

Em geral não é uma boa prática

Jack não permite esse tipo de acesso



UFRPE

Programação OO: construindo uma classe

- Como a gente constrói uma classe?

Classe Fraction

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;
    /** Constructs a (reduced) fraction from the given numerator and denominator. */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce(); // reduces the fraction
        return this; // returns the base address of the new object
    }
    // Reduces this fraction.
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {let numerator = numerator / g; let denominator = denominator / g;}
        return;
    }
    // Computes the greatest common divisor of the given integers.
    function int gcd(int a, int b) {
        var int r;
        while (~(b = 0)) { // applies Euclid's algorithm.
            let r = a - (b * (a / b)); // r = remainder of the integer division a/b
            let a = b; let b = r; }
        return a;
    }
} // More Fraction code in next slide
```

Programação OO: construindo uma classe

- Os campos ou atributos da classe

Classe Fraction

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;
    /** Constructs a (reduced) fraction from the given numerator and denominator. */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce(); // reduces the fraction
        return this; // returns the base address of the new object
    }
    // Reduces this fraction.
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {let numerator = numerator / g; let denominator = denominator / g;}
        return;
    }
    // Computes the greatest common divisor of the given integers.
    function int gcd(int a, int b) {
        var int r;
        while (~(b = 0)) { // applies Euclid's algorithm.
            let r = a - (b * (a / b)); // r = remainder of the integer division a/b
            let a = b; let b = r; }
        return a;
    }
} // More Fraction code in next slide
```

Programação OO: construindo uma classe

- Sub-rotinas: construtor, method, function

Classe Fraction

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;
    /** Constructs a (reduced) fraction from the given numerator and denominator. */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce(); // reduces the fraction
        return this; // returns the base address of the new object
    }
    // Reduces this fraction.
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {let numerator = numerator / g; let denominator = denominator / g;}
        return;
    }
    // Computes the greatest common divisor of the given integers.
    function int gcd(int a, int b) {
        var int r;
        while (~(b = 0)) { // applies Euclid's algorithm.
            let r = a - (b * (a / b)); // r = remainder of the integer division a/b
            let a = b; let b = r; }
        return a;
    }
} // More Fraction code in next slide
```

Programação OO: construindo uma classe

Classe Fraction

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;
    /** Constructs a (reduced) fraction from the given numerator and denominator. */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce(); // reduces the fraction
        return this; // returns the base address of the new object
    }
    // Reduces this fraction.
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {let numerator = numerator / g; let denominator = denominator / g;}
        return;
    }
    // Computes the greatest common divisor of the given integers.
    function int gcd(int a, int b) {
        var int r;
        while (~(b = 0)) {
            let r = a - (b * (a / b)); // applies Euclid's algorithm.
            let a = b; let b = r; } // r = remainder of the integer division a/b
        return a;
    }
} // More Fraction code in next slide
```

Uma sub-rotina deve terminar com um comando de retorno.

Programação OO: construindo uma classe

- Construtores são sub-rotinas que criam novos objetos

Classe Fraction

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;
    /** Constructs a (reduced) fraction from the given numerator and denominator. */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce(); // reduces the fraction
        return this; // returns the base address of the new object
    }
    // Reduces this fraction.
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {let numerator = numerator / g; let denominator = denominator / g;}
        return;
    }
    // Computes the greatest common divisor of the given integers.
    function int gcd(int a, int b) {
        var int r;
        while (~(b = 0)) { // applies Euclid's algorithm.
            let r = a - (b * (a / b)); // r = remainder of the integer division a/b
            let a = b; let b = r; }
        return a;
    }
} // More Fraction code in next slide
```

Programação OO: construindo uma classe

Classe Fraction

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;
    /** Constructs a (reduced) fraction from the given numerator and denominator. */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce(); // reduces the fraction
        return this; // returns the base address of the new object
    }
    // Reduces this fraction.
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {let numerator = numerator / g; let denominator = denominator / g;}
        return;
    }
    // Computes the greatest common divisor of the given integers.
    function int gcd(int a, int b) {
        var int r;
        while (~(b = 0)) { // applies Euclid's algorithm.
            let r = a - (b * (a / b)); // r = remainder of the integer division a/b
            let a = b; let b = r; }
        return a;
    }
} // More Fraction code in next slide
```

- **this**: uma referência ao objeto atual
- O construtor deve retornar a referência do objeto recém-criado.

Programação OO: construindo uma classe

Classe Fraction

- Exemplo do método soma.
- Não precisamos saber como foi feita.
- Só precisamos saber como utilizá-la.

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;

    /** Returns the sum of this fraction and the other one. */
    method Fraction plus(Fraction other) {
        var int sumNumerators;
        let sumNumerators = (numerator * other.getDenominator()) +
                           (other.getNumerator() * denominator);
        return Fraction.new(sumNumerators, denominator * other.getDenominator());
    }

    // More Fraction methods follow.
} // Fraction class
```

Programação OO: construindo uma classe

Fraction class

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;

    /** Prints this fraction in the format x/y. */
    method void print() {
        do Output.printInt(numerator);
        do Output.printString("/");
        do Output.printInt(denominator);
        return;
    }

    // More Fraction methods follow.
} // Fraction class
```



Programação OO: construindo uma classe

Fraction class

```
/** Represents the Fraction type and related operations. */  
class Fraction {  
    field int numerator, denominator;  
  
    /** Disposes this fraction. */  
    method void dispose() {  
        do Memory.deAlloc(this); // uses an OS routine to recycle the object's memory.  
        return;  
    }  
} // Fraction class
```

- **Garbage collection:**
 - Jack não possui garbage collection
 - Objetos precisam ser desalocados explicitamente
 - Boa prática: toda a classe que possui um construtor deve também possuir um método *dispose*.



UFRPE

Programação OO: representação de um objeto

Fraction class

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;

    /** Constructs a (reduced) fraction from the given numerator and denominator */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce();
        return this;
    }

    // More Fraction methods
}
```

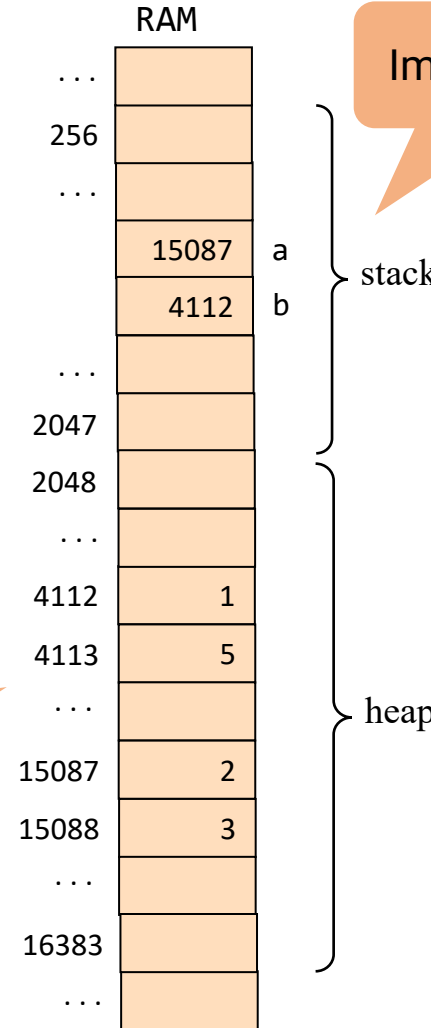
The compiled constructor's code includes OS calls that allocate and manage RAM space for representing the new object

Issues:

- allocating memory
- de-allocating memory (handled by the compiler and the OS)

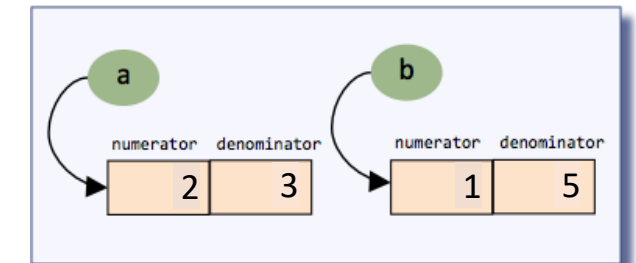
Client code

```
...
var Fraction a, b;
...
let a = Fraction.new(2,3);
let b = Fraction.new(1,5);
...
```



Implementação

The client's view



Programação OO: representação de um objeto

Fraction class

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;

    /** Constructs a (reduced) fraction from the given numerator and denominator */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce();
        return this;
    }

    // More Fraction methods
}
```

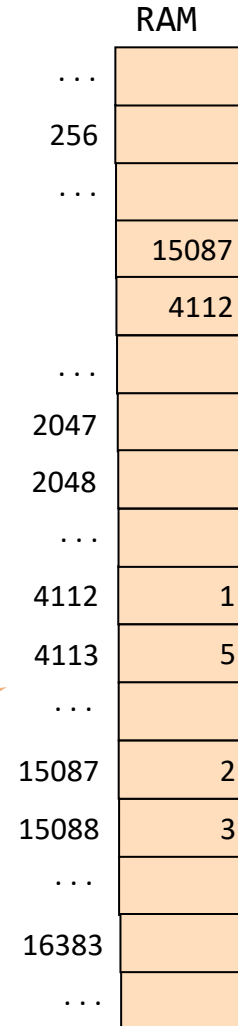
The compiled constructor's code includes OS calls that allocate and manage RAM space for representing the new object

Issues:

- allocating memory
- de-allocating memory (handled by the compiler and the OS)

Client code

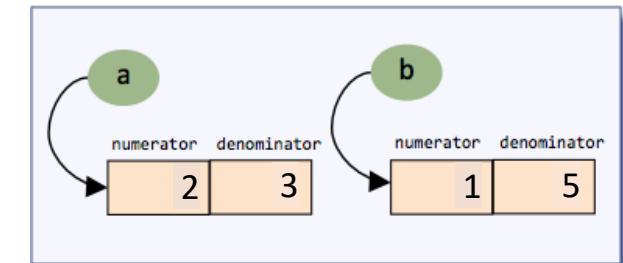
```
...
var Fraction a, b;
...
let a = Fraction.new(2,3);
let b = Fraction.new(1,5);
...
```



Implementação

stack

The client's view



heap

- Stack: é destinada à memória estática, usada para chamadas de função e variáveis locais
- Heap: é destinado à memória dinâmica, para dados de tamanho desconhecido ou que precisam persistir por mais tempo do que uma única chamada de função.



Programação OO: representação de um objeto

Fraction class

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;

    /** Constructs a (reduced) fraction from the given numerator and denominator */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce();
        return this;
    }

    // More Fraction methods
}
```

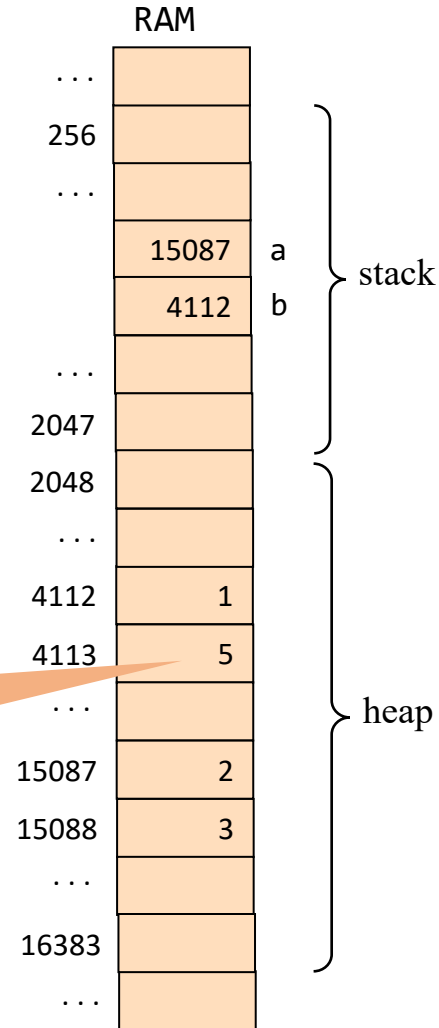
The compiled constructor's code includes OS calls that allocate and manage RAM space for representing the new object

Client code

```
...
var Fraction a, b, c;
...
let a = Fraction.new(2,3);
let b = Fraction.new(1,5);
...
```

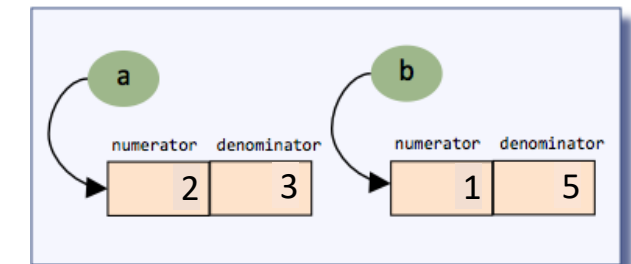
Issues:

- allocating memory
- de-allocating memory (handled by the compiler and the OS)



Abstração

The client's view



Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Listas

- Definição de Lista
 - valor **null**, ou
 - valor, seguido por uma lista

Exemplos:

null

(5, null)

(3, (5, null))

(2, (3, (5, null)))

Abreviado como:

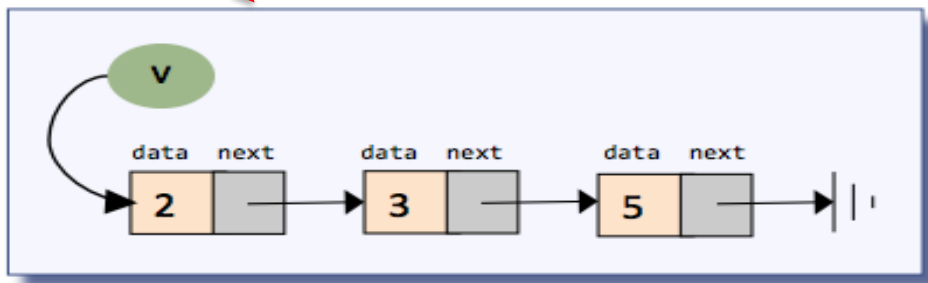
()

(5)

(3, 5)

(2, 3, 5)

Exemplo visual de lista



The list (2, (3, (5, null))),
commonly abbreviated as (2, 3, 5)

Listas

List API (partial)

```
/** Represents a linked list of integers. */
class List {

    /** Creates a List */
    constructor List new(int car, List cdr)

    /** Prints this list */
    method void print() {

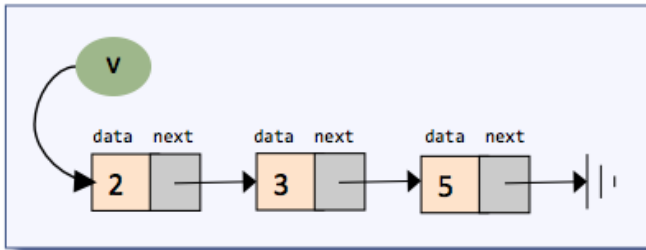
    /** Disposes this list */
    method void dispose() {

    }
}
```



```
// Builds, prints, and disposes a list
class Main {
    function void main() {
        // Creates and manipulates the list (2,(3,(5,null))),
        // commonly referred to as (2,3,5)
        var List v;
        let v = List.new(5,null);
        let v = List.new(3,v);
        let v = List.new(2,v);
        do v.print();
        do v.dispose(); // disposes the list
        return;
    }
}
```

List example



The list (2, (3, (5, null))),
commonly abbreviated as (2, 3, 5)

Listas: criação

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.
```

client code

```
...  
var List v;  
  
let v = List.new(5,null);  
  
let v = List.new(3, v);  
  
let v = List.new(2, v);  
  
...
```

Listas: criação

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /* Creates a List. */  
    constructor List new(int car, List cdr) {  
        let data = car;  
        let next = cdr;  
        return this;  
    }  
  
    // More List methods  
}
```

client code

```
...  
var List v;  
  
let v = List.new(5,null);  
  
let v = List.new(3, v);  
  
let v = List.new(2, v);  
  
...
```

v

Listas: criação

List class

```
/** Represents a linked list of integers. */
class List {
  field int data;    // a list consists of a data field,
  field List next;   // followed by a list.

  /** Creates a List. */
  constructor List new(int car, List cdr) {
    let data = car;
    let next = cdr;
    return this;
  }

  // More List methods
}
```

client code

```
...
var List v;
let v = List.new(5,null);
let v = List.new(3, v));
let v = List.new(2, v));
...
```

v

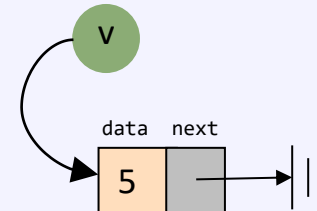
Listas: criação

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /* Creates a List. */  
    constructor List new(int car, List cdr) {  
        let data = car;  
        let next = cdr;  
        return this;  
    }  
  
    // More List methods  
}
```

client code

```
...  
var List v;  
let v = List.new(5, null);  
let v = List.new(3, v);  
let v = List.new(2, v);  
...
```



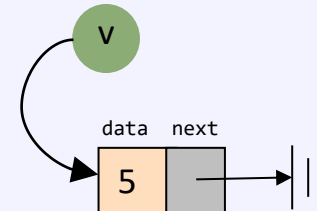
Listas: criação

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /* Creates a List. */  
    constructor List new(int car, List cdr) {  
        let data = car;  
        let next = cdr;  
        return this;  
    }  
  
    // More List methods  
}
```

client code

```
...  
var List v;  
let v = List.new(5,null);  
let v = List.new(3, v);  
let v = List.new(2, v);  
...
```



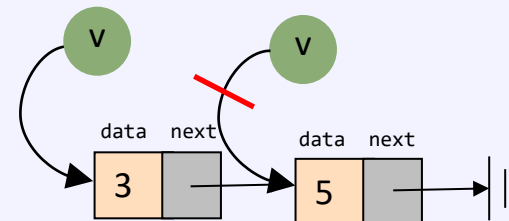
Listas: criação

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /* Creates a List. */  
    constructor List new(int car, List cdr) {  
        let data = car;  
        let next = cdr;  
        return this;  
    }  
  
    // More List methods  
}
```

client code

```
...  
var List v;  
let v = List.new(5, null);  
let v = List.new(3, v);  
let v = List.new(2, v);  
...
```



Listas: criação

List class

```
/** Represents a linked list of integers. */
class List {
  field int data;    // a list consists of a data field,
  field List next;  // followed by a list.

  /** Creates a List. */
  constructor List new(int car, List cdr) {
    let data = car;
    let next = cdr;
    return this;
  }

  // More List methods
}
```

client code

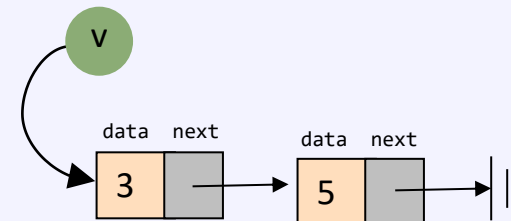
```
...
var List v;

let v = List.new(5, null);

let v = List.new(3, v);

let v = List.new(2, v);

...
```



Listas: criação

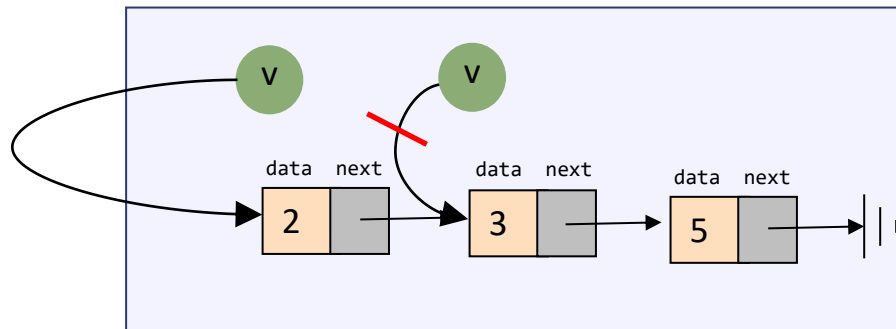
List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Creates a List. */  
    constructor List new(int car, List cdr) {  
        let data = car;  
        let next = cdr;  
        return this;  
    }  
  
    // More List methods  
}
```

client code

```
...  
var List v;  
  
let v = List.new(5, null);  
let v = List.new(3, v);  
let v = List.new(2, v);  
...  

```



Listas: criação

List class

```
/** Represents a linked list of integers. */
class List {
  field int data;    // a list consists of a data field,
  field List next;  // followed by a list.

  /** Creates a List. */
  constructor List new(int car, List cdr) {
    let data = car;
    let next = cdr;
    return this;
  }

  // More List methods
}
```

client code

```
...
var List v;

let v = List.new(5,null);

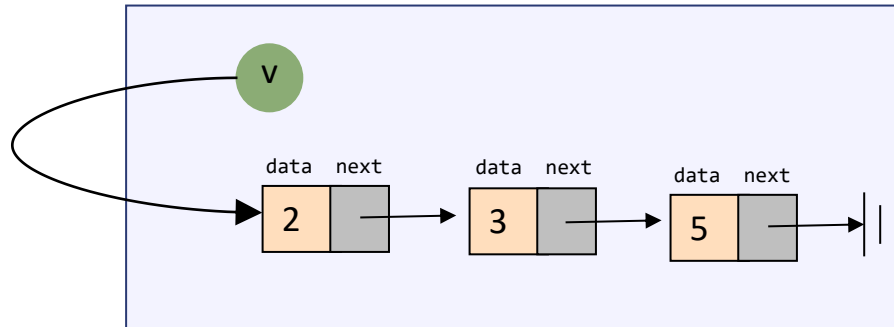
let v = List.new(3, v);

let v = List.new(2, v);

...
```

equivalent to:

```
var List v;
let v = List.new(5,null);
let v = List.new(2,List.new(3,v));
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Prints this list. */  
    method void print() {
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;  // followed by a list.

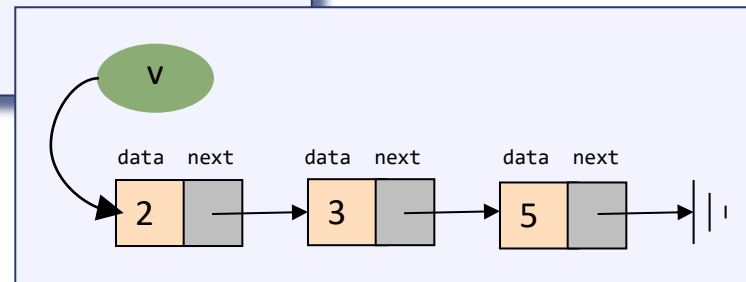
    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list

        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }
        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

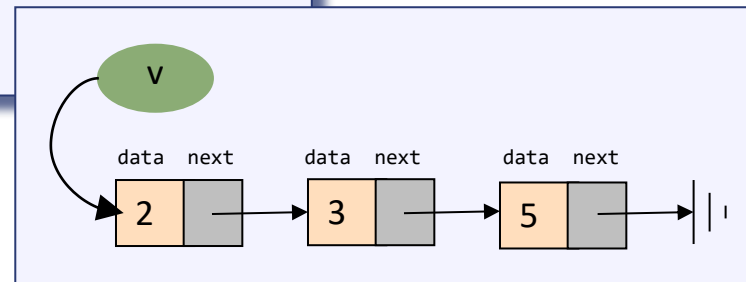
```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;   // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;   // initializes it to the first item of this list
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }
        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

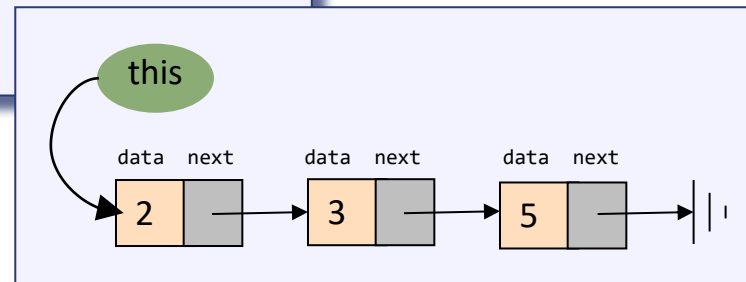
```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;  // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }
        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;   // followed by a list.

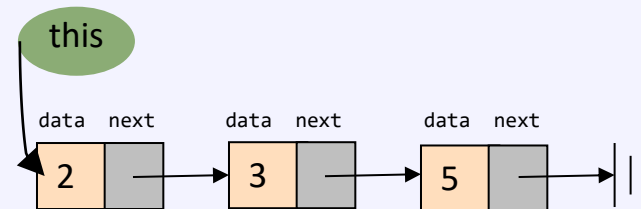
    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;   // initializes it to the first item of this list

        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }
        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



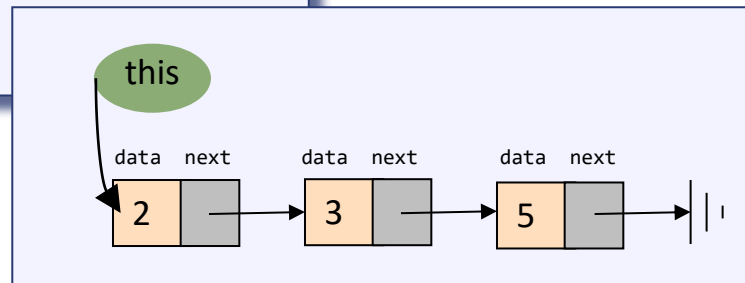
Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Prints this list. */  
    method void print() {  
        var List current;    // creates a List variable and  
        let current = this;  // initializes it to the first item of this list  
  
        while (~(current = null)) {  
            do Output.printInt(current.getData());  
            do Output.printChar(32);    // prints a space  
            do current = current.getNext();  
        }  
        return;  
    }  
  
    // More list methods  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.print();  
...
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;   // followed by a list.

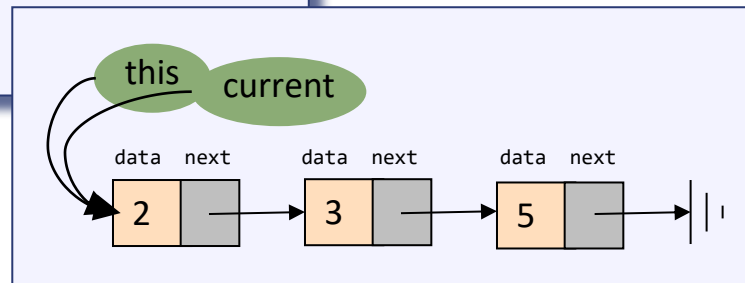
    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list

        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }
        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;  // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list

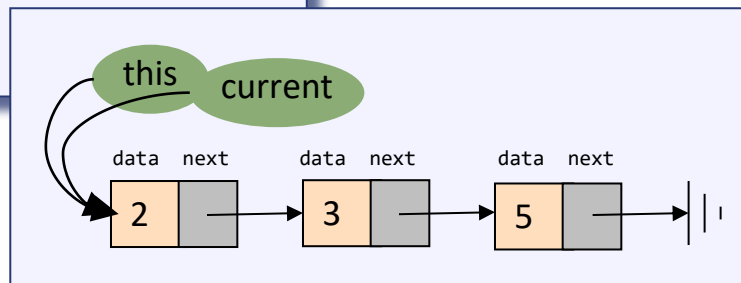
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }

        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;  // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list

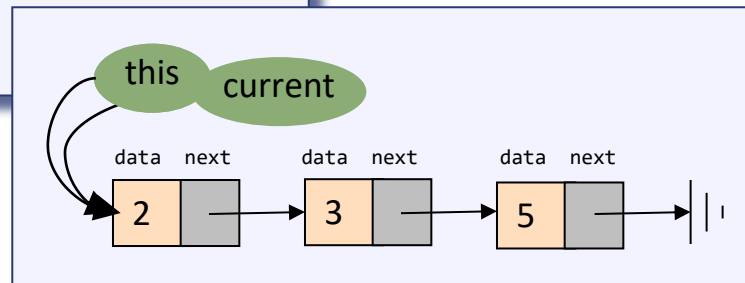
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }

        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;  // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list

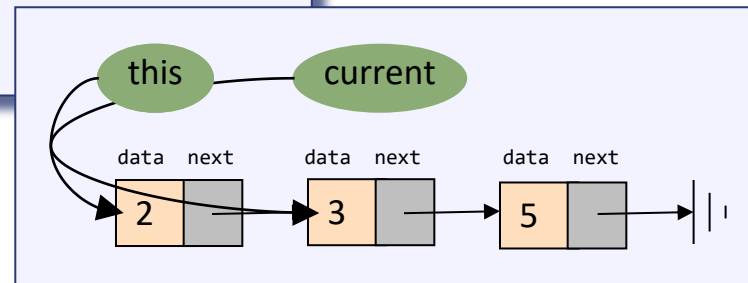
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }

        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



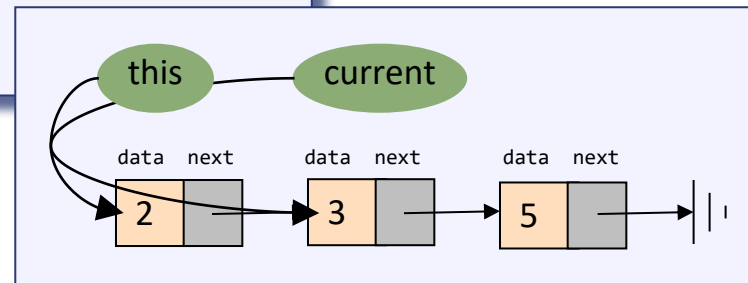
Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Prints this list. */  
    method void print() {  
        var List current;    // creates a List variable and  
        let current = this;  // initializes it to the first item of this list  
  
        while (~(current = null)) {  
            do Output.printInt(current.getData());  
            do Output.printChar(32);    // prints a space  
            do current = current.getNext();  
        }  
  
        return;  
    }  
  
    // More list methods  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.print();  
...
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;  // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list

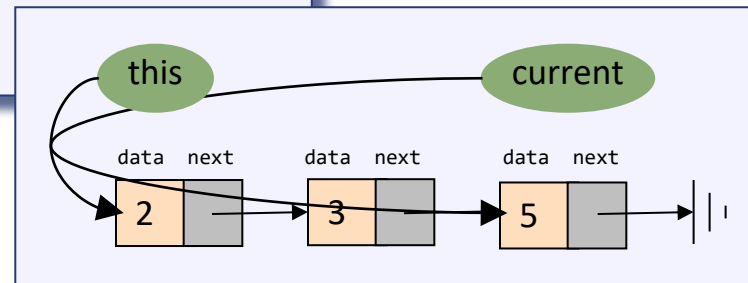
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }

        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;   // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list

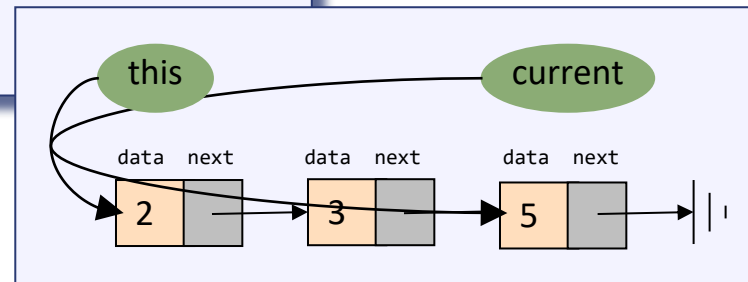
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }

        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;  // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list

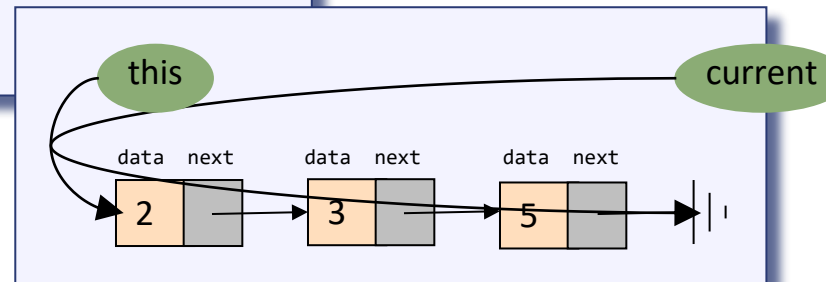
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }

        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;   // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list

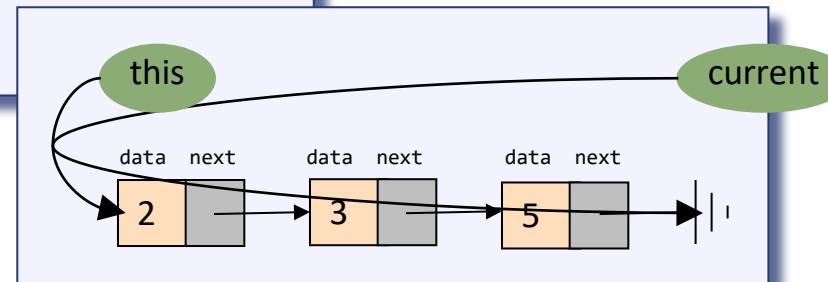
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }

        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

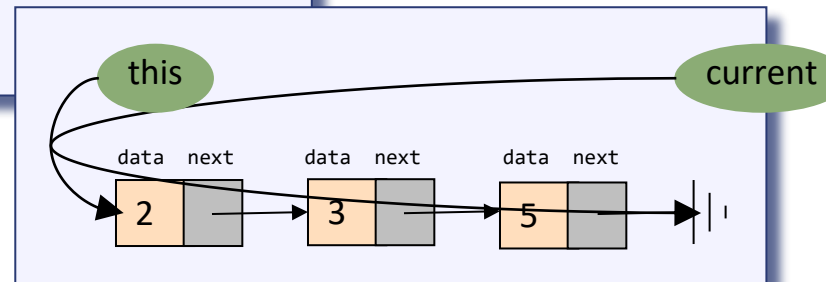
```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;   // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }
        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: Com realizar acesso sequencial ?

List class

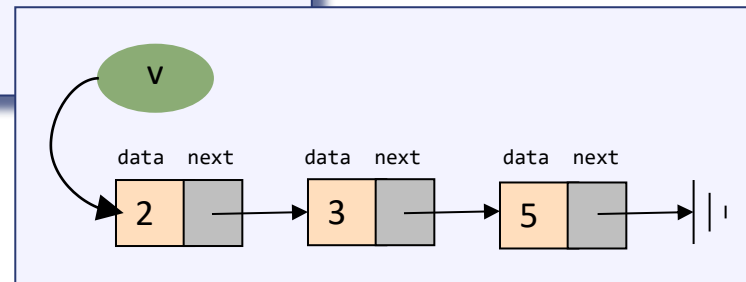
```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;  // followed by a list.

    /** Prints this list. */
    method void print() {
        var List current;    // creates a List variable and
        let current = this;  // initializes it to the first item of this list
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32);    // prints a space
            do current = current.getNext();
        }
        return;
    }

    // More list methods
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.print();
...
```



Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;   // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```



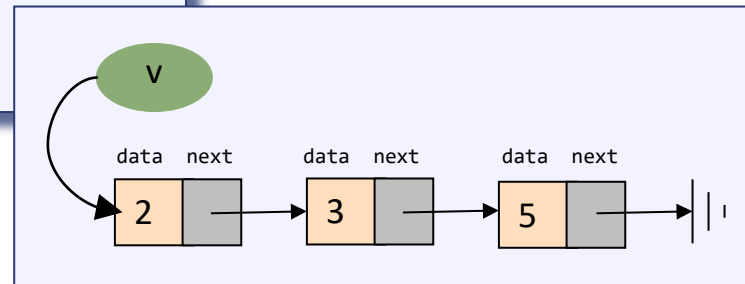
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



Listas: acesso recursivo

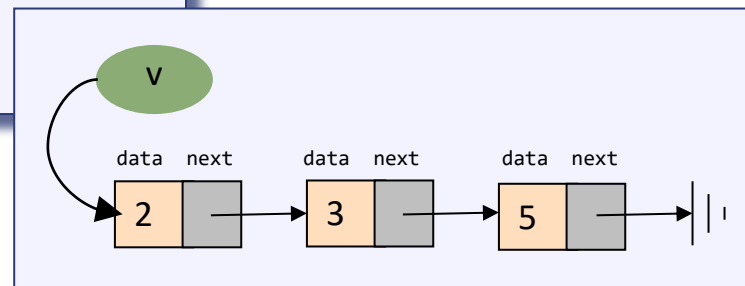
List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;  // followed by a list.

    /** Disposes this list */
    // by recursively disposing its tail
    method void dispose() {
        if (~(next = null)) {
            do next.dispose();
        }
        // Uses an OS routine to recycle this list
        do Memory.deAlloc(this);
        return;
    }
}
```

client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();
...
```



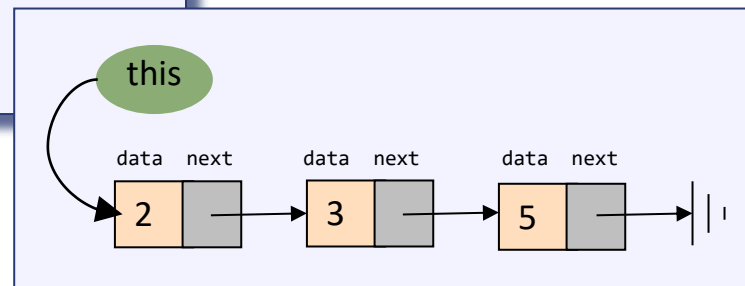
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



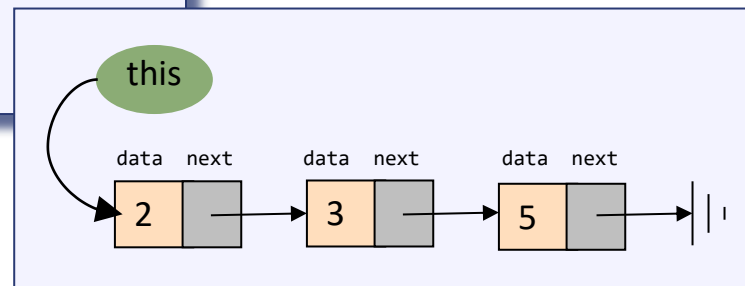
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



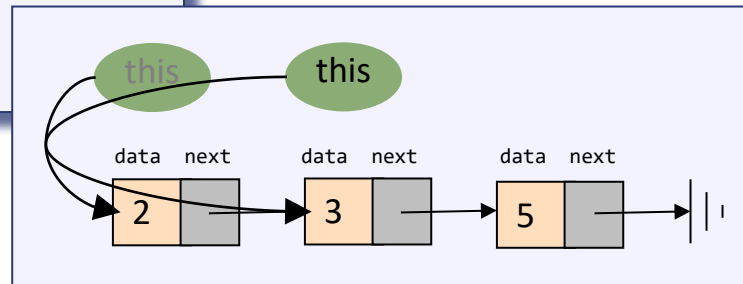
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



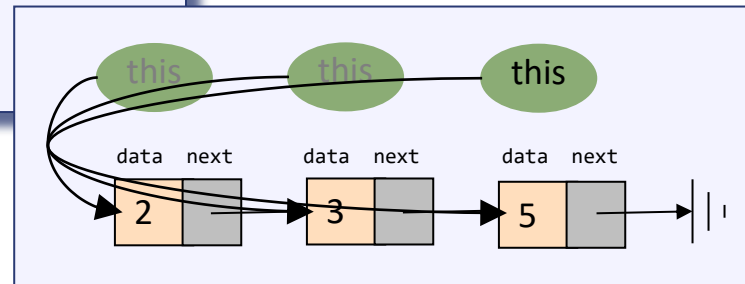
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



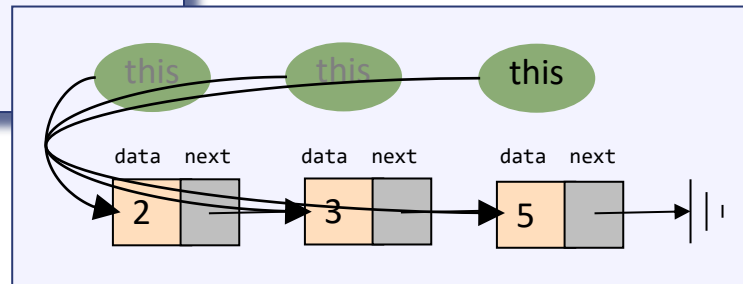
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



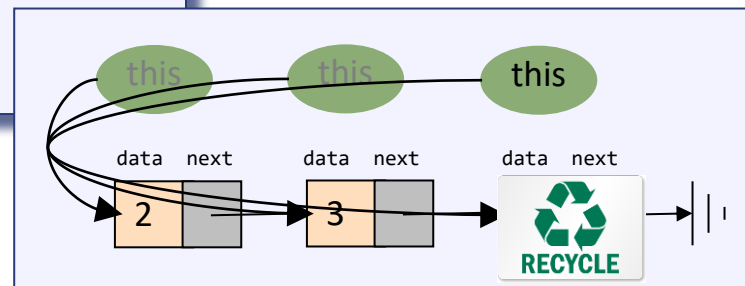
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;   // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



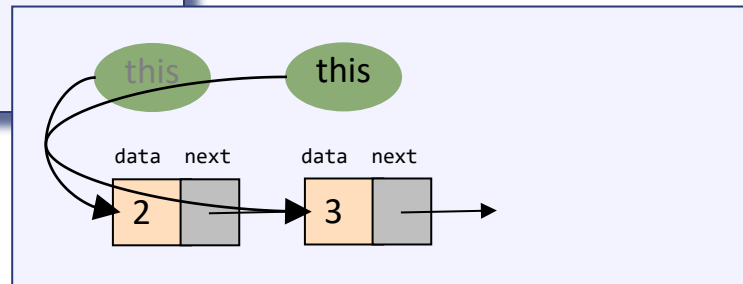
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



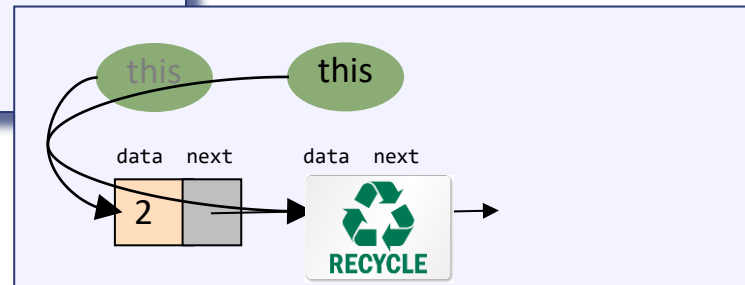
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



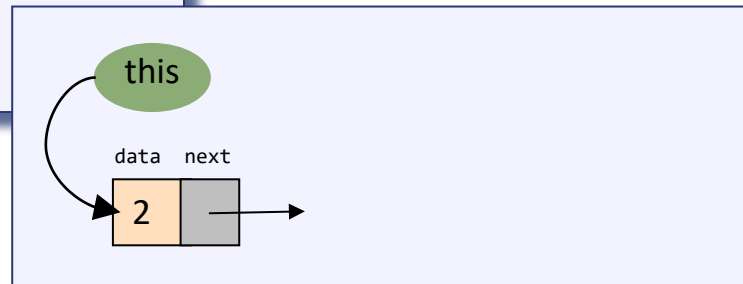
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;  // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



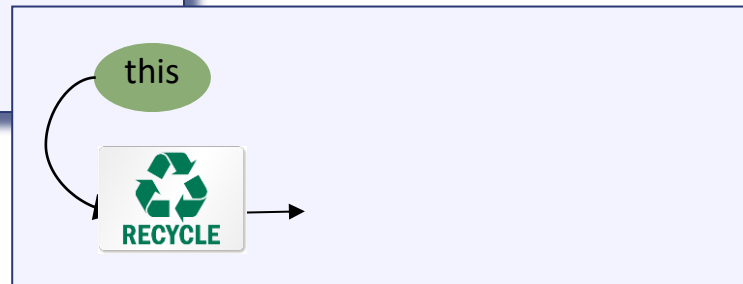
Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;   // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```



Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;   // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```

this

Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */  
class List {  
    field int data;    // a list consists of a data field,  
    field List next;   // followed by a list.  
  
    /** Disposes this list */  
    // by recursively disposing its tail  
    method void dispose() {  
        if (~(next = null)) {  
            do next.dispose();  
        }  
        // Uses an OS routine to recycle this list  
        do Memory.deAlloc(this);  
        return;  
    }  
}
```

client code

```
var List v;  
// builds the list (2, 3, 5)  
...  
do v.dispose();  
...
```

this

Listas: acesso recursivo

List class

```
/** Represents a linked list of integers. */
class List {
    field int data;    // a list consists of a data field,
    field List next;   // followed by a list.

    /** Disposes this list */
    // by recursively disposing its tail
    method void dispose() {
        if (~(next = null)) {
            do next.dispose();
        }
        // Uses an OS routine to recycle this list
        do Memory.deAlloc(this);
        return;
    }
}
```

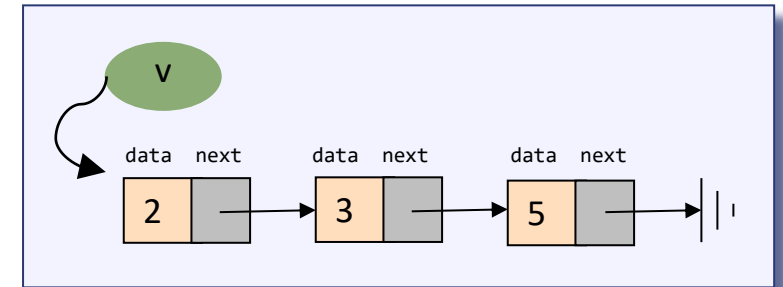
client code

```
var List v;
// builds the list (2, 3, 5)
...
do v.dispose();
...
```

v

Recapitulando...

- Vimos exemplos de como:
 - Construir uma lista
 - Processar uma lista, sequencialmente
 - Processar uma lista, recursivamente
 - Liberar uma lista
- Quem e o que fazem a mágica funcionar?
 - Quando um objeto é construído ou liberado, o sistema operacional é responsável por alocar e desalocar, respectivamente, o espaço de memória RAM necessário.
 - O processamento de lista é feito através da manipulação de ponteiros
 - A manipulação de ponteiros é feita através do código que é gerado pelo compilador.



Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Sintaxe: espaços em branco / comentários

```
/** Procedural processing example */
class Main {
    /* Inputs some numbers and computes their average */
    function void main() {
        var Array a;
        var int length;
        var int i, sum;
        let length = Keyboard.readInt("How many numbers? ");
        let a = Array.new(length); // constructs the array
        let i = 0;
        while (i < length) {
            let a[i] = Keyboard.readInt("Enter a number: ");
            let sum = sum + a[i];
            let i = i + 1;
        }
        ...
    }
}
```

- Elementos de sintaxe:
 - **espaços em branco / comentários**
 - palavras-chave ou palavras reservadas
 - símbolos
 - constantes
 - identificadores

Caracteres de espaço, caracteres de nova linha e comentários **são ignorados**.

Os seguintes formatos de comentários são suportados:

```
// Comment to end of line
/* Comment until closing */
/** API documentation comment */
```



UFRPE

Sintaxe: palavras-chave

```
/** Procedural processing example */
class Main {
    /* Inputs some numbers and computes their average */
    function void main() {
        var Array a;
        var int length;
        var int i, sum;
        let length = Keyboard.readInt("How many numbers? ");
        let a = Array.new(length); // constructs the array
        let i = 0;
        while (i < length) {
            let a[i] = Keyboard.readInt("Enter a number: ");
            let sum = sum + a[i];
            let i = i + 1;
        }
        ...
    }
}
```

- Elementos de sintaxe:
 - espaços em branco / comentários
 - **palavras-chave ou palavras reservadas**
 - símbolos
 - constantes
 - identificadores

<u>class, constructor, method, function</u>	<u>Program components</u>
<u>int, boolean, char, void</u>	<u>Primitive types</u>
<u>var, static, field</u>	<u>Variable declarations</u>
<u>let, do, if, else, while, return</u>	<u>Statements</u>
<u>true, false, null</u>	<u>Constant values</u>
<u>this</u>	<u>Object reference</u>

- Palavras predefinidas que possuem significados especiais e são reservadas para propósitos específicos dentro da sintaxe da linguagem.
- Essas palavras formam uma parte fundamental da gramática da linguagem

Sintaxe: palavras-chave

```
/** Procedural processing example */
class Main {
    /* Inputs some numbers and computes their average */
    function void main() {
        var Array a;
        var int length;
        var int i, sum;
        let length = Keyboard.readInt("How many numbers? ");
        let a = Array.new(length); // constructs the array
        let i = 0;
        while (i < length) {
            let a[i] = Keyboard.readInt("Enter a number: ");
            let sum = sum + a[i];
            let i = i + 1;
        }
        ...
    }
}
```

- Elementos de sintaxe:
 - espaços em branco / comentários
 - palavras-chave ou palavras reservadas
 - símbolos
 - constantes
 - identificadores

<u>class, constructor, method, function</u>	<u>Program components</u>
<u>int, boolean, char, void</u>	<u>Primitive types</u>
<u>var, static, field</u>	<u>Variable declarations</u>
<u>let, do, if, else, while, return</u>	<u>Statements</u>
<u>true, false, null</u>	<u>Constant values</u>
<u>this</u>	<u>Object reference</u>

Não podem ser usadas por programadores como identificadores (nomes de variáveis, nomes de funções ou nomes de classes) em seu código.

Sintaxe: símbolos

```
/** Procedural processing example */  
class Main {  
    /* Inputs some numbers and computes their average */  
    function void main() {  
        var Array a;  
        var int length;  
        var int i, sum;  
        let length = Keyboard.readInt("How many numbers? ");  
        let a = Array.new(length); // constructs the array  
        let i = 0;  
        while (i < length) {  
            let a[i] = Keyboard.readInt("Enter a number: ");  
            let sum = sum + a[i];  
            let i = i + 1;  
        }  
        ...  
    }  
}
```

- Elementos de sintaxe:
 - espaços em branco / comentários
 - palavras-chave ou palavras reservadas
 - **símbolos**
 - constantes
 - identificadores

Não podemos
utilizar para outra
finalidade

- () Used for grouping arithmetic expressions and for enclosing parameter-lists and argument-lists;
- [] Used for array indexing;
- { } Used for grouping program units and statements;

- , Variable list separator;
- ; Statement terminator;
- = Assignment and comparison operator;
- . Class membership;
- + - * / & | ~ < > Operators.

Sintaxe: constantes

```
/** Procedural processing example */
class Main {
    /* Inputs some numbers and computes their average */
    function void main() {
        var Array a;
        var int length;
        var int i, sum;
        let length = Keyboard.readInt("How many numbers? ");
        let a = Array.new(length); // constructs the array
        let i = 0;
        while (i < length) {
            let a[i] = Keyboard.readInt("Enter a number: ");
            let sum = sum + a[i];
            let i = i + 1;
        }
        ...
    }
}
```

- Elementos de sintaxe:
 - espaços em branco / comentários
 - palavras-chave ou palavras reservadas
 - símbolos
 - **constantes**
 - identificadores

- Constantes inteiras são valores no intervalo de 0 a 32767.
- Números inteiros negativos, como -13, não são constantes, mas sim expressões que consistem em um operador de subtração unário aplicado a uma constante inteira.
- Constantes de string são delimitadas por aspas duplas (") e podem conter qualquer caractere, exceto nova linha ou aspas duplas, que podem ser obtidas chamando as funções do sistema operacional `String.newLine()` e `String.doubleQuote()`.
- Constantes booleanas podem ser `true` ou `false`.
- A constante `null` representa uma referência nula.

Sintaxe: identificadores

```
/** Procedural processing example */
class Main {
    /* Inputs some numbers and computes their average */
    function void main() {
        var Array a;
        var int length;
        var int i, sum;
        let length = Keyboard.readInt("How many numbers? ");
        let a = Array.new(length); // constructs the array
        let i = 0;
        while (i < length) {
            let a[i] = Keyboard.readInt("Enter a number: ");
            let sum = sum + a[i];
            let i = i + 1;
        }
        ...
    }
}
```

- Elementos de sintaxe:
 - espaços em branco / comentários
 - palavras-chave ou palavras reservadas
 - símbolos
 - constantes
 - **identificadores**

Caracteres que
podemos usar para
criar variáveis

- Os identificadores são compostos por sequências arbitrariamente longas de letras (A a Z, a a z), dígitos (0 a 9) e "_".
- O primeiro caractere deve ser uma letra ou "_".
- A linguagem diferencia maiúsculas de minúsculas: x e X são tratados como identificadores diferentes.

Recapitulando...

```
/** Procedural processing example */  
class Main {  
    /* Inputs some numbers and computes their average */  
    function void main() {  
        var Array a;  
        var int length;  
        var int i, sum;  
        let length = Keyboard.readInt("How many numbers? ");  
        let a = Array.new(length); // constructs the array  
        let i = 0;  
        while (i < length) {  
            let a[i] = Keyboard.readInt("Enter a number: ");  
            let sum = sum + a[i];  
            let i = i + 1;  
        }  
        ...  
    }  
}
```

- Elementos de sintaxe:
 - espaços em branco / comentários
 - palavras-chave ou palavras reservadas
 - símbolos
 - constantes
 - identificadores



Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Tipos de dados

```
/** Procedural processing example */
class Main {
  /* Inputs some numbers and computes their average */
  function void main() {
    var Array a;
    var int length;
    var int i, sum;

    let length = Keyboard.readInt("How many numbers? ");
    let a = Array.new(length); // constructs the array
    let i = 0;
    while (i < length) {
      let a[i] = Keyboard.readInt("Enter a number: ");
      let sum = sum + a[i];
      let i = i + 1;
    }
    ...
  }
}
```

- Tipos primitivos

- int:
valores inteiros não-negativos em notação complemento à dois, i.e. um inteiro no intervalo 0,..., 32767
- boolean:
true OU false
- char:
valores inteiros representando caracteres

- Tipos de classes:

- OS types: String, Array
- User-defined types: Fraction, List, ...



Conjunto de caracteres do Hack

key	code
(space)	32
!	33
“	34
#	35
\$	36
%	37
&	38
‘	39
(	40
)	41
*	42
+	43
,	44
-	45
.	46
/	47

key	code
0	48
1	49
...	...
9	57

:	58
;	59
<	60
=	61
>	62
?	63
@	64

key	code
A	65
B	66
C	...
...	...
Z	90

[	91
/	92
]	93
^	94
_	95
`	96

key	code
a	97
b	98
c	99
...	...
z	122

{	123
	124
}	125
~	126

key	code
newline	128
backspace	129
left arrow	130
up arrow	131
right arrow	132
down arrow	133
home	134
end	135
Page up	136
Page down	137
insert	138
delete	139
esc	140
f1	141
...	...
f12	152

Conversão de tipos

Caracteres e números inteiros podem ser convertidos uns nos outros, conforme necessário:

```
var char c; var String s;  
let c = 65; // 'A'  
let c = 'A'; // Not supported by the Jack language  
let s = "A";  
let c = s.charAt(0); // 'A', ok
```

Jack não permite esse tipo de atribuição

Um número inteiro pode ser atribuído a uma variável de referência, caso em que é tratado como um endereço de memória:

```
var Array arr; // creates a pointer variable  
let arr = 5000; // ok...  
let arr[100] = 17; // sets memory address 5100 to 17
```

Um objeto pode ser convertido em um Array e vice-versa:

```
var Array arr;  
let arr = Array.new(2);  
let arr[0] = 2;  
let arr[1] = 5;  
var Fraction x; // a Fraction object has two int fields: numerator and denominator.  
let x = arr; // sets x to the base address of the memory block representing the array [2,5]  
do x.print() // prints 2/5 (using the print method of the Fraction class)
```

Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Classes

- **Class** = unidade básica de compilação
- A classe **Foo** é armazenada em um arquivo **Foo.jack** separado
- A primeira letra do nome de uma classe deve possuir um caracter maiúsculo

Foo.jack file

```
class Foo {  
    field variable declarations  
    static variable declarations  
    subroutine declarations  
}
```

deve preceder a
declaração das
subrotinas.

Classes

- **Class** = unidade básica de compilação
- A classe **Foo** é armazenada em um arquivo **Foo.jack** separado
- A primeira letra do nome de uma classe deve possuir um caracter maiúsculo

Foo.jack file

```
class Foo {  
    field variable declarations  
    static variable declarations  
    subroutine declarations  
}
```

deve preceder a
declaração das
subrotinas.

construtores,
métodos e
funções

Classes utilitárias

- Math class API (exemplo)
 - Provê várias operações matemáticas

- function int **abs**(int x): returns the absolute value of x.
- function int **multiply**(int x, int y): returns the product of x and y.
- function int **divide**(int x, int y): returns the integer part of x/y.
- function int **min**(int x, int y): returns the minimum of x and y.
- function int **max**(int x, int y): returns the maximum of x and y.
- function int **sqrt**(int x): returns the integer part of the square root of x.

- Uma classe que contém apenas funções (“métodos estáticos”) pode ser chamada uma classe utilitária.
- Oferece uma “biblioteca de serviços”.



Classes que representam entidades (objetos)

- Exemplos: Fraction, List, String, ...
- Uma classe que contém pelo menos um método
- Utilizado para representar um tipo de objeto e operações sobre este objeto
- Tipicamente contém atributos e métodos

Boa prática:

Não misturar funcionalidades da biblioteca com a representação do objeto na mesma classe.



Mau exemplo típico...

Fraction class

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;

    /** Constructs a (reduced) fraction from the given numerator and denominator. */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce(); // reduces the fraction
        return this; // returns the base address of the new object
    }

    // Reduces this fraction.
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {let numerator = numerator / g; let denominator = denominator / g;}
        return;
    }

    // Computes the greatest common divisor of the given integers.
    function int gcd(int a, int b) {
        var int r;
        while (~(b = 0)) { // applies Euclid's algorithm.
            let r = a - (b * (a / b)); // r = remainder of the integer division a/b
            let a = b; let b = r; }
        return a;
    }

    // More Fraction methods
}
```

Mau exemplo típico...

Fraction class

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;

    /** Constructs a (reduced) fraction from the given numerator and denominator. */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce(); // reduces the fraction
        return this; // returns the base address of the new object
    }

    // Reduces this fraction.
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {let numerator = numerator / g; let denominator = denominator / g;}
        return;
    }

    // Computes the greatest common divisor of the given integers.
    function int gcd(int a, int b) {
        var int r;
        while (~(b = 0)) { // applies Euclid's algorithm.
            let r = a - (b * (a / b)); // r = remainder of the integer division a/b
            let a = b; let b = r; }
        return a;
    }

    // More Fraction methods
}
```

Depois de um refatoramento...

Fraction class

```
/** Represents the Fraction type and related operations. */
class Fraction {
    field int numerator, denominator;

    /** Constructs a (reduced) fraction from the given numerator and denominator. */
    constructor Fraction new(int x, int y) {
        let numerator = x; let denominator = y;
        do reduce(); // reduces the fraction
        return this; // returns the base address of the new object
    }

    // Reduces this fraction.
    method void reduce() {
        var int g;
        let g = Math.gcd(numerator, denominator);
        if (g > 1) {let numerator = numerator / g; let denominator = denominator / g;
        return;
        }

        // More Fraction methods
    } // End of Fraction class
```

```
/** Library of mathematical functions. */
class Math {
    ...

    // Computes the greatest common divisor of the given integers.
    function int gcd(int a, int b) {
        var int r;
        while (~(b = 0)) {
            let r = a - (b * (a / b));
            let a = b; let b = r;
        }
        return a;
    }

    // More math functions
} // End of Math class
```

Aplicação Jack

- Um programa Jack, ou uma aplicação, é uma coleção de uma ou mais classes Jack, onde uma delas deve ser nomeada como **Main**
- A classe **Main** precisa ter necessariamente uma função chamada **main**
- A entrada de qualquer programa: `Main.main`



Biblioteca padrão Jack / S0

Jack code

```
// Inputs some numbers and computes their average
class Main {
  function void main() {
    var Array a;
    var int length;
    var int i, sum;
    let length = Keyboard.readInt("How many numbers? ");
    let a = Array.new(length); // constructs the array
    let i = 0;
    while (i < length) {
      let a[i] = Keyboard.readInt("Enter a number: ");
      let sum = sum + a[i];
      let i = i + 1;
    }
    do Output.printString("The average is ");
    do Output.printInt(sum / length);
    return;
  }
}
```

- **Propósito do S0:**
 - resolve os *gaps* entre os programas de alto nível e o hardware do host
 - provê implementação eficiente de funções utilizadas com grande frequência.
 - provê implementações eficientes dos tipos abstratos de dados (ADTs)
- **Implementação do S0:**
 - uma coleção de classes
 - similar a biblioteca padrão de Java



UFRPE

Biblioteca padrão Jack / S0

<i>OS class</i>	<i>Services</i>
Math	Common mathematical operations: <code>multiply(int,int)</code> , <code>sqrt(int)</code> , etc.
String	Represents string objects and related methods: <code>length()</code> , <code>charAt(int)</code> , etc.
Array	Represents array objects and related operations: <code>new(int)</code> , <code>dispose()</code> .
Output	Supports text output to the screen: <code>printString(String)</code> , <code>printInt(int)</code> , <code>println()</code> , etc.
Screen	Supports graphics output to the screen: <code>drawPixel(int,int)</code> , <code>setColor(boolean)</code> , <code>drawCircle(int,int,int)</code> , etc.
Keyboard	Supports input from the keyboard: <code>readLine(String)</code> , <code>readInt(String)</code> , etc.
Memory	Facilitates access to the host RAM: <code>peek(int)</code> , <code>poke(int,int)</code> , <code>alloc(int)</code> , <code>deAlloc(Array)</code> .
Sys	Supports execution-related services: <code>halt()</code> , <code>wait(int)</code> , etc.

API completa do S0: livro e website do Nand2Tetris

Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Classes

```
class Foo {  
    field variable declarations  
    static variable declarations  
    subroutine declarations  
}
```



Subrotinas

Subroutine declaration

```
constructor | method | function  type subroutineName (parameter-list) {  
    local variable declarations  
    statements  
}
```

- **Subrotinas em Jack:**
 - Construtores: criam novos objetos
 - Métodos: operam sobre o objeto corrente
 - Funções: métodos estáticos
- **Tipos de subrotinas e valores de retorno**
 - O tipo de método ou função pode ser void, um tipo primitivo de dados, ou o nome de uma classe
 - Cada subrotina deve terminar com um **return** ou **return valor**

Construtores

Constructor declaration

```
constructor ClassName constructorName (parameter-list) {  
    local variable declarations  
    statements  
}
```

- **Construtores:**
 - 0, 1, ou mais em uma classe
 - Nome comum: new
- O tipo do construtor deve ter o mesmo nome da classe
- o construtor deve retornar uma referência para um objeto do mesmo tipo da classe

```
class Point {  
    field int x;  
    field int y;  
    ...  
    /* Creates a Point. */  
    constructor Point new(int ax, int ay) {  
        let x = ax;  
        let y = ay;  
        return this;  
    }  
    ...  
}
```



Variáveis

- Tipos de variáveis:
 - variáveis *field* (atributo):
propriedades do objeto, pode ser manipulado pelos construtores da classe e pelos métodos
 - variáveis estáticas:
variáveis no nível da classe, podem ser manipuladas pelas subrotinas da classe
 - variáveis locais:
utilizadas pela subrotinas para realizar computações locais
 - parâmetros:
utilizadas para passar valores para a subrotinas.
Comportam-se como variáveis locais
- As variáveis devem ser:
 - declaradas antes de serem utilizadas
 - tipadas

```
/** Represents a Point object. */  
class Point {  
    field int x, y;  
    static int pointCount;  
    ...  
    method int bla(int z) {  
        int foo;  
        let foo = z * (x + y);  
        return foo;  
    }  
    ...  
}
```

Variáveis

Variable kind	Description	Declared in	Scope
static variables	<code>static type varName1, varName2, ... ;</code> Only one copy of each static variable exists, and this copy is shared by all the object instances of the class (like <i>private static variables</i> in Java)	class declaration	The class in which they are declared.
field variables	<code>field type varName1, varName2, ... ;</code> Every object (instance of the class) has a private copy of the field variables (like <i>member variables</i> in Java)	class declaration	The class in which they are declared, except for functions, where they are undefined.
local variables	<code>var type varName1, varName2, ... ;</code> Local variables are created just before the subroutine starts running and are disposed when it returns (like <i>local variables</i> in Java)	subroutine declaration	The subroutine in which they are declared.
parameter variables	<code>type varName1, varName2, ...</code> Used to pass arguments to the subroutine. Treated like local variables whose values are initialized “from the outside”, just before the subroutine starts running.	subroutine signature	The subroutine in which they are declared.

Declarações (*Statements*)

Statement	Syntax	Description
let	<code>let <i>varName</i> = <i>expression</i>;</code> or <code>let <i>varName</i>[<i>expression1</i>] = <i>expression2</i>;</code>	An assignment operation (where <i>varName</i> is either single-valued or an array). The variable kind may be <i>static</i> , <i>local</i> , <i>field</i> , or <i>parameter</i> .
if	<code>if (<i>expression</i>) { <i>statements1</i> } else { <i>statements2</i> }</code>	Typical <i>if</i> statement with an optional <i>else</i> clause. The curly brackets are mandatory even if <i>statements</i> is a single statement.
while	<code>while (<i>expression</i>) { <i>statements</i> }</code>	Typical <i>while</i> statement. The curly brackets are mandatory even if <i>statements</i> is a single statement.
do	<code>do <i>function-or-method-call</i>;</code>	Used to call a function or a method for its effect, ignoring the returned value.
return	<code>Return <i>expression</i>;</code> or <code>return;</code>	Used to return a value from a subroutine. The second form must be used by functions and methods that return a void value. Constructors must return the expression <code>this</code> .

Expressões

A *Jack expression* is one of the following:

- A *constant*
- A *variable name* in scope. The variable may be *static*, *field*, *local*, or *parameter*
- The *this* keyword, denoting the current object (cannot be used in functions)
- An *array element* using the syntax *Arr[expression]*, where *Arr* is a variable name of type *Array* in scope
- A *subroutine call* that returns a non-void type
- An expression prefixed by one of the unary operators - or ~:
 - *expression*: arithmetic negation
 - ~ *expression*: boolean negation (bit-wise for integers)
- An expression of the form *expression op expression* where *op* is one of the following binary operators:

+	-	*	/	Integer arithmetic operators
&				Boolean And and Boolean Or (bit-wise for integers) operators
<	>	=		Comparison operators
- (*expression*): An expression in parenthesis

Chamadas de subrotinas

Examples:

```
class Foo {  
    ...  
    method void f() {  
        var Bar b;      // declares a local variable of class type Bar  
        var int i;       // declares a local variable of primitive type int  
        ...  
        do g();          // calls method g of the current class on this object  
        do Foo.p(3);     // calls function p of the current class  
        do Bar.h();      // calls function h of class Bar  
        let b = Bar.r(); // calls function or constructor r of class Bar  
        do b.q();        // calls method q of class Bar on the b object  
        let i = w(b.s(), Foo.t()); // calls method w on this object. The arguments are  
                                   // the results of calling method s on object b,  
                                   // and function or constructor t of class Foo  
        ...  
    }  
}
```

- Sintaxe das chamadas de subrotina:
nomeDaSubrotina(lista-de-argumentos)
- A quantidade e o tipo dos argumentos devem ser **compatíveis** com os parâmetros das subrotinas
- Cada argumento pode ser uma expressão de complexidade ilimitada



UFRPE

Strings

Examples:

```
...  
var String s; // Creates an object variable (pointer), initialized to null  
var char c;   // Creates a primitive variable, initialized to zero  
...  
// Suppose we want s to refer to the string "Hello World!":  
let s = String.new(12);  
// Followed by a loop that uses String's appendChar method to set s to "Hello World!"  
// (loop code not shown)  
  
// Alternatively, the Jack compiler allows using:  
let s = "Hello World"; // "syntactic sugar"  
  
// Accessing some character within a string:  
let c = s.charAt(6);    // Sets c to the numeric value representing 'W'.
```

A API da classe *String* fornece todas as funções aceitas pela biblioteca, consulte-a no site nand2tetris.org



Arrays

Examples:

```
...  
var Array arr;  
var String helloWorld;  
let helloWorld = "Hello World!"  
...  
let arr = Array.new(4);  
let arr[0] = 12;  
let arr[1] = false;  
let arr[2] = Fraction.new(314/100);  
let arr[3] = helloWorld;  
...
```

- Os arrays Jack são:
 - instâncias da classe Array do SO
 - não tipado
 - uni-dimensional
- Arrays multi-dimensionais podem ser obtidos utilizando-se arrays de arrays



UFRPE

Características peculiares da linguagem Jack

- A palavra-chave `let`:
 - deve ser utilizadas em atribuições como: `let x = 0;`
- A palavra-chave `do`:
 - deve ser utilizada para chamar um método ou uma função fora de uma expressão: `do reduce();`
- O corpo de uma declaração deve estar entre chaves, mesmo que contenha apenas uma única declaração: `if (a > 0) {return a;} else {return -a;}`
- Toda subrotina deve terminar com a palavra-chave `return`
- Não há precedência de operadores:
 - o valor resultante da seguinte expressão é indeterminado: `2+3*4`
 - para garantir a precedência deve-se utilizar parênteses: `2+(3*4)`
- A linguagem é fracamente tipada

Algumas destas concessões diminuem a complexidade do compilador

Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

A parte mais legal do curso!

Vamos ver a nossa aplicação rodando em cima de toda a infraestrutura que a gente começou a construir

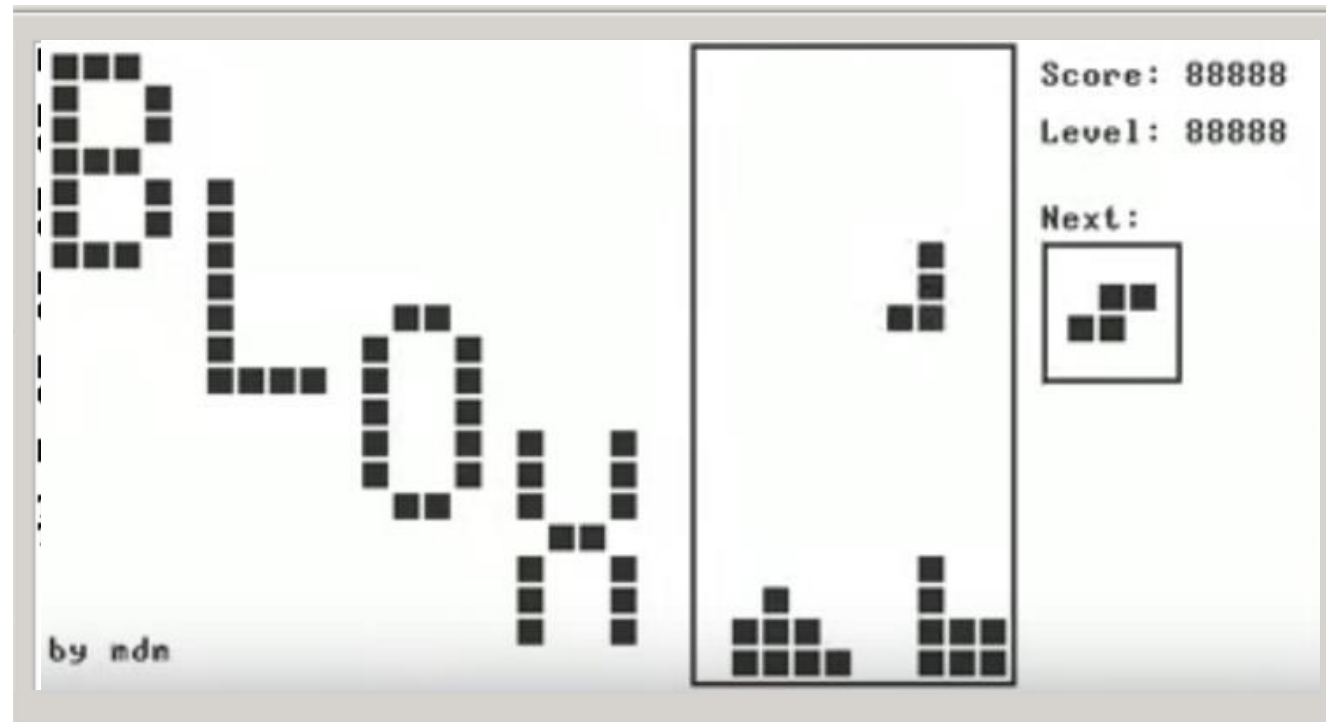


UFRPE

Algumas aplicações: Estatísticas

```
Enter the students data, ending with 'Q':  
Name: DAN  
Grade: 90  
Name: PAUL  
Grade: 80  
Name: LISA  
Grade: 100  
Name: ANN  
Grade: 90  
Name: Q  
The grades average is 90  
The student with the highest grade is LISA
```

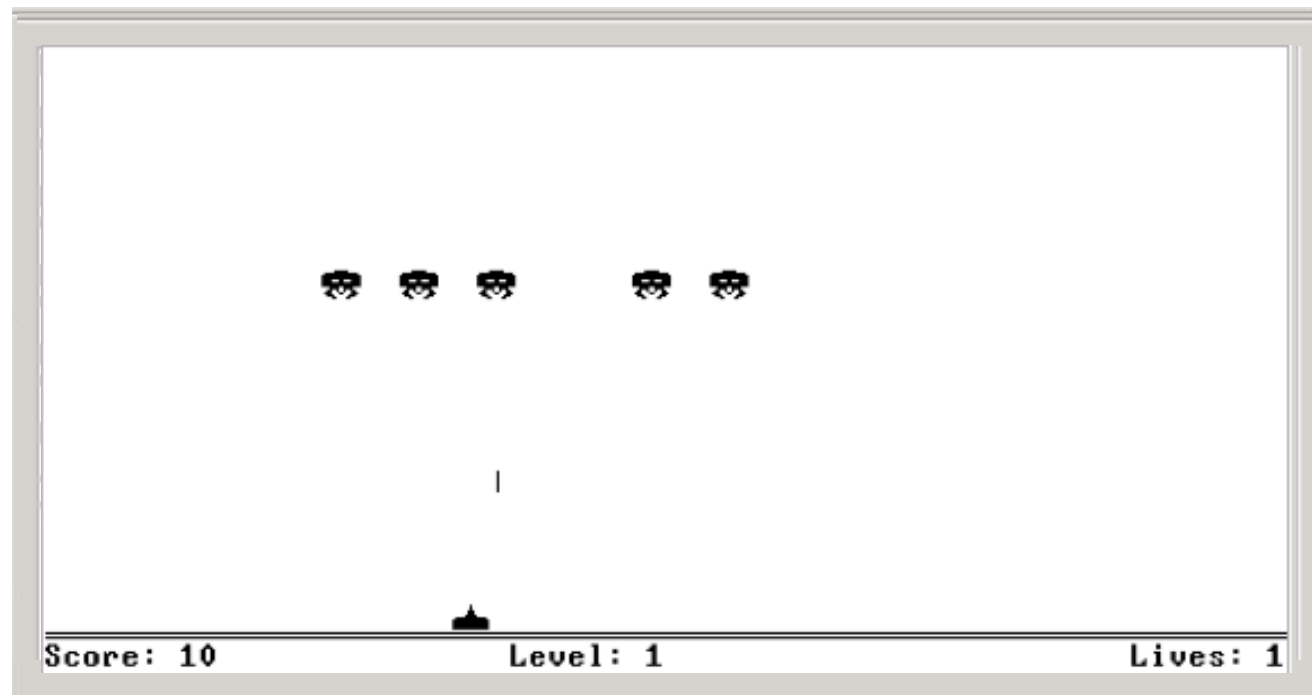
Algumas aplicações: Tetris (por Domink Migge)



Algumas aplicações: *Bouncing Ball* (por Gavin Stewart)

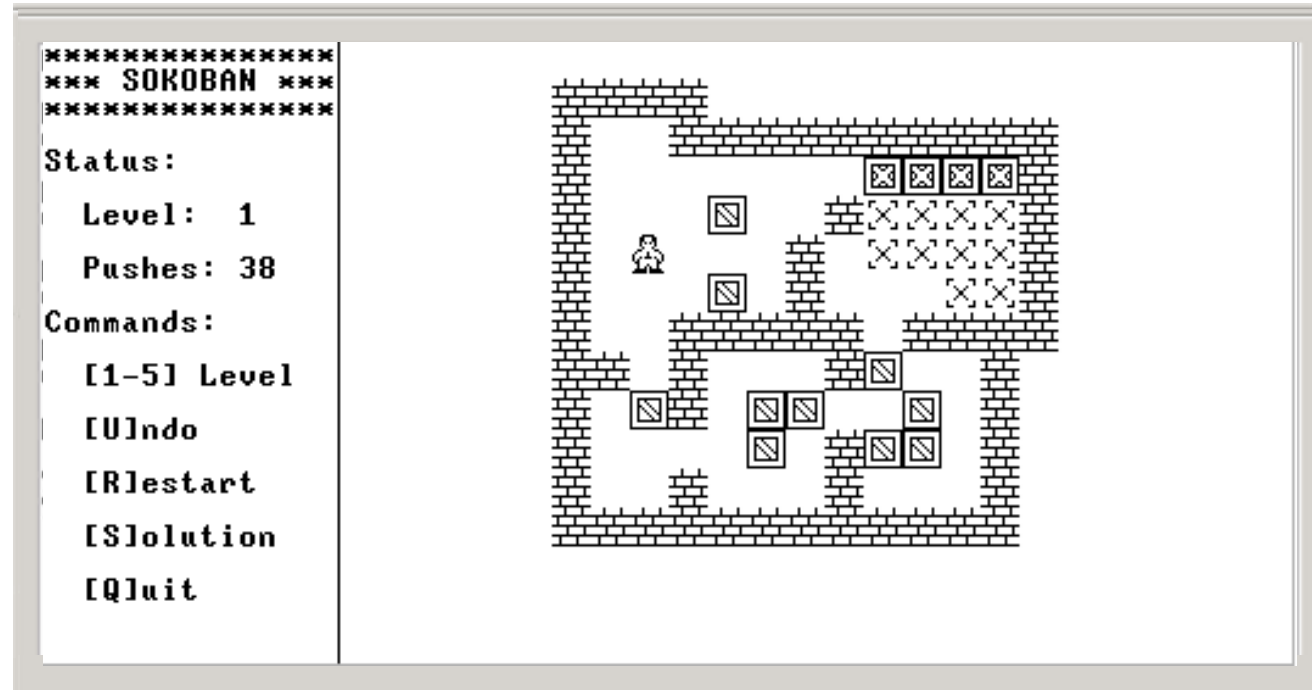


Algumas aplicações: *Space Invaders* (por *Ran Navok*)



Algumas aplicações: Sokoban

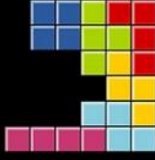
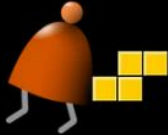
(por Golan Parashi)



Outras aplicações

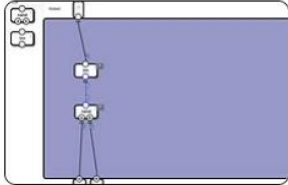
From Nand to Tetris

Building a Modern Computer From First Principles



- Home
- Projects
- Book
- Software
- Demos
- License
- Cool Stuff
- Team
- Stay in Touch
- Q&A

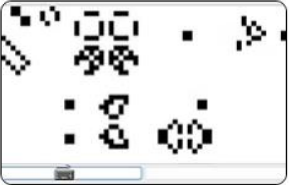
Cool Stuff



The Nand Game
Plus a visual gate editor,
by Olav Junker Kjær



Dyno Adventure
By Erik Umble



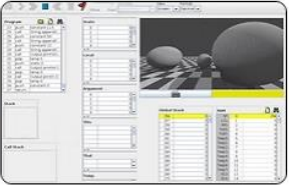
Conway's Life Game
By Robert Woodhead



Turtle Graphics Console
Actually, a logo interpreter...
By Jay Winston



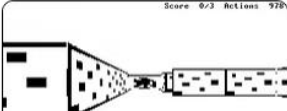
Hack in FPGA
By Michael Schröder



Ray Tracing on Hack/Jack
By Alex Quach



Leo Game
by Joseph
in Jack



Score 0/3 Actions 920



40 to DOHUT

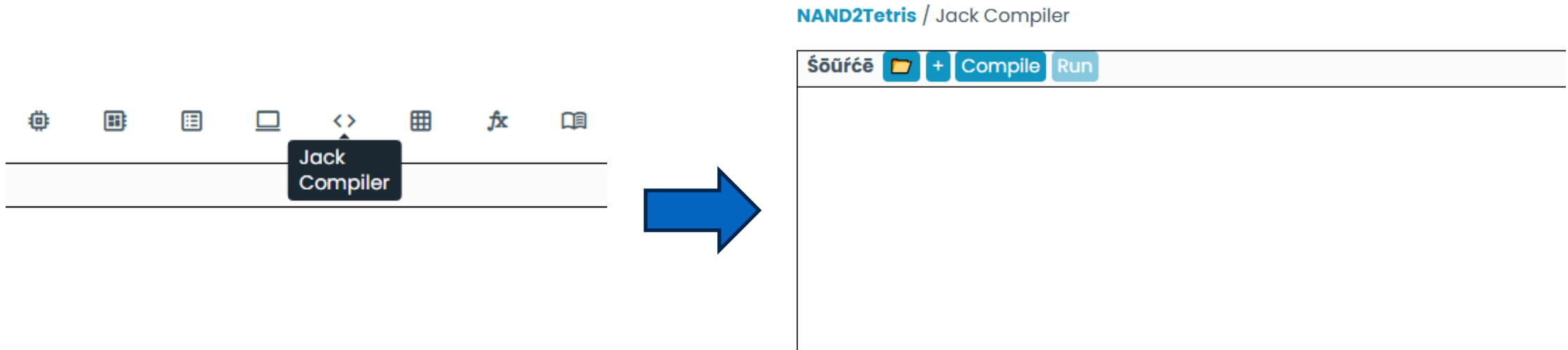
Desenvolvendo uma aplicação Jack

- Coloque todos os arquivos da aplicação em uma pasta, cujo nome seja igual ao nome do aplicativo
- ***Escreva ou edite*** suas classes Jack utilizando um editor de textos padrão



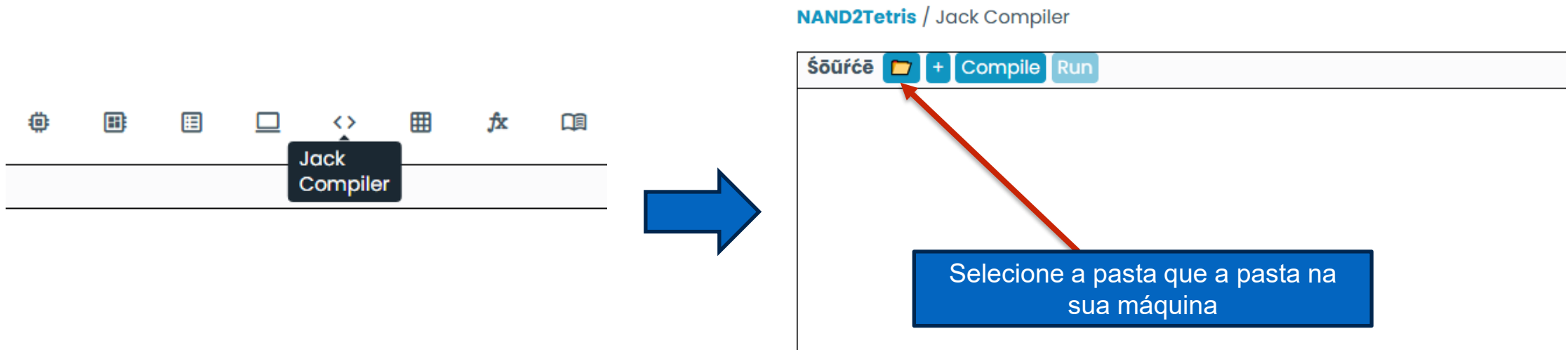
Desenvolvendo uma aplicação Jack

- **Compile** seus arquivos Jack / diretório utilizando a ferramenta online **JackCompiler**



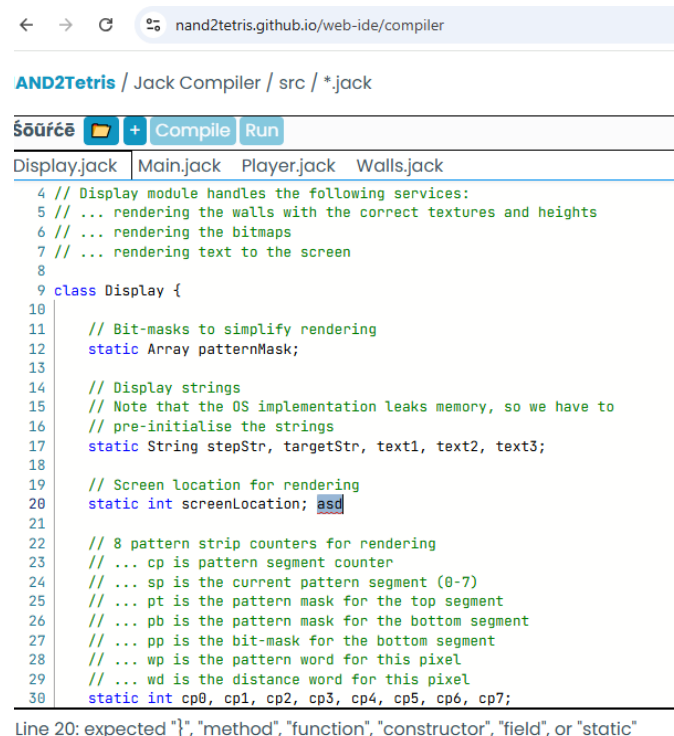
Desenvolvendo uma aplicação Jack

- **Compile** seus arquivos Jack / diretório utilizando a ferramenta online **JackCompiler**



Desenvolvendo uma aplicação Jack

- **Compile** a aplicação: Todos os arquivos Jack são compilados. Se houver um erro de sintaxe, o botão Compile estará desativada e o compilador irá relatar o erro.



The screenshot shows the Jack Compiler web interface at `nand2tetris.github.io/web-ide/compiler`. The file `Display.jack` is selected. The code is as follows:

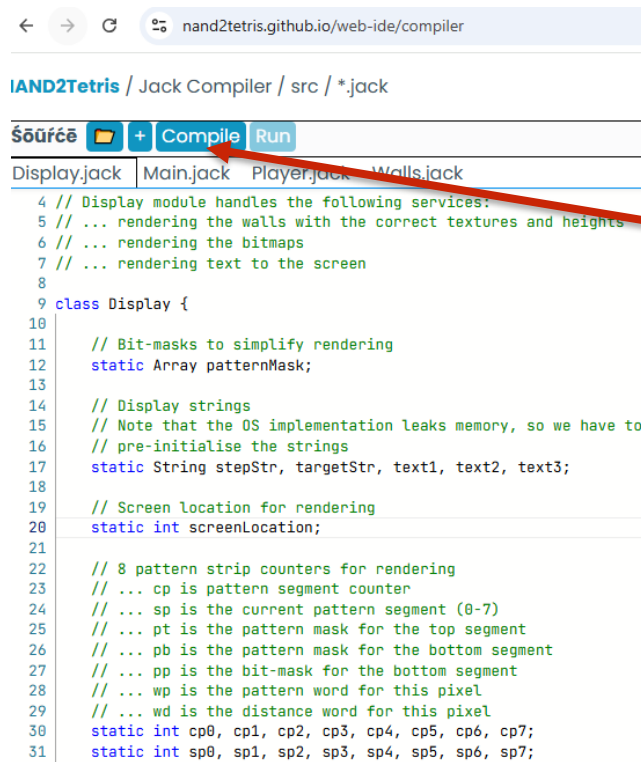
```
4 // Display module handles the following services:
5 // ... rendering the walls with the correct textures and heights
6 // ... rendering the bitmaps
7 // ... rendering text to the screen
8
9 class Display {
10
11     // Bit-masks to simplify rendering
12     static Array patternMask;
13
14     // Display strings
15     // Note that the OS implementation leaks memory, so we have to
16     // pre-initialise the strings
17     static String stepStr, targetStr, text1, text2, text3;
18
19     // Screen location for rendering
20     static int screenLocation; asd
21
22     // 8 pattern strip counters for rendering
23     // ... cp is pattern segment counter
24     // ... sp is the current pattern segment (0-7)
25     // ... pt is the pattern mask for the top segment
26     // ... pb is the pattern mask for the bottom segment
27     // ... pp is the bit-mask for the bottom segment
28     // ... wp is the pattern word for this pixel
29     // ... wd is the distance word for this pixel
30     static int cp0, cp1, cp2, cp3, cp4, cp5, cp6, cp7;
```

An error message at the bottom states: "Line 20: expected '}', 'method', 'function', 'constructor', 'field', or 'static'". A red arrow points from this message to a blue box labeled "Descrição do erro".

Descrição do erro

Desenvolvendo uma aplicação Jack

- **Compile** a aplicação: Se todos os códigos estiverem corretos o botão Compile estará habilitado e você poderá clicar-lo

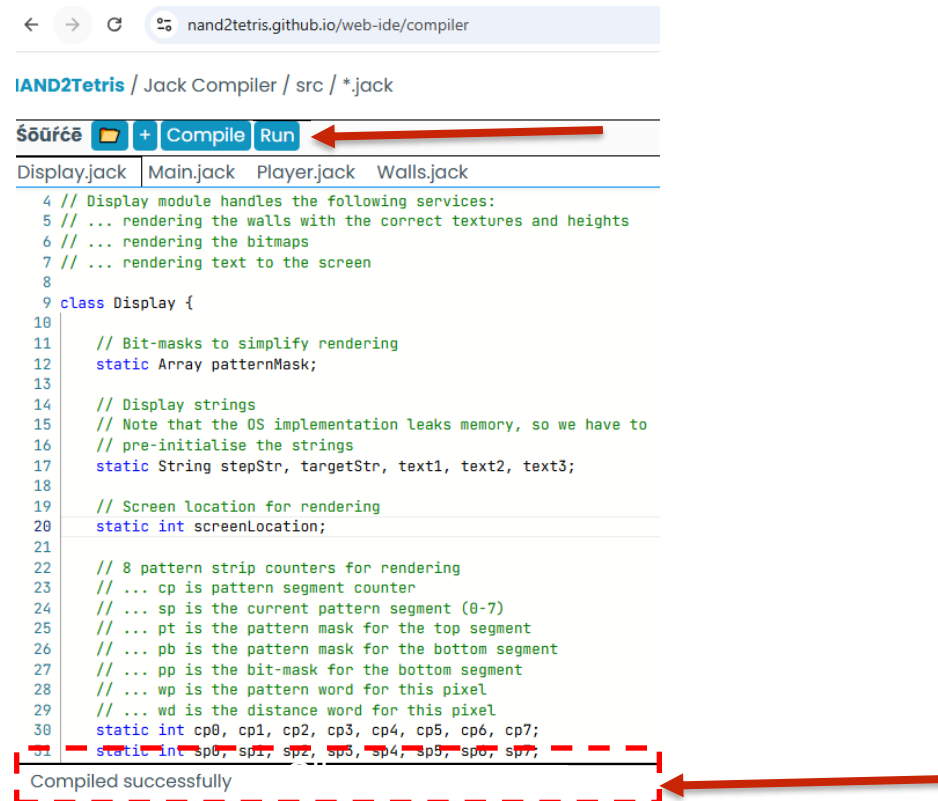


```
4 // Display module handles the following services:
5 // ... rendering the walls with the correct textures and heights
6 // ... rendering the bitmaps
7 // ... rendering text to the screen
8
9 class Display {
10
11     // Bit-masks to simplify rendering
12     static Array patternMask;
13
14     // Display strings
15     // Note that the OS implementation leaks memory, so we have to
16     // pre-initialise the strings
17     static String stepStr, targetStr, text1, text2, text3;
18
19     // Screen location for rendering
20     static int screenLocation;
21
22     // 8 pattern strip counters for rendering
23     // ... cp is pattern segment counter
24     // ... sp is the current pattern segment (0-7)
25     // ... pt is the pattern mask for the top segment
26     // ... pb is the pattern mask for the bottom segment
27     // ... pp is the bit-mask for the bottom segment
28     // ... wp is the pattern word for this pixel
29     // ... wd is the distance word for this pixel
30     static int cp0, cp1, cp2, cp3, cp4, cp5, cp6, cp7;
31     static int sp0, sp1, sp2, sp3, sp4, sp5, sp6, sp7;
```

Clique

Desenvolvendo uma aplicação Jack

- **Compile** a aplicação: Você então verá uma mensagem de sucesso no canto inferior e o botão run estará habilitado



The screenshot shows the nand2tetris web IDE compiler interface. The browser address bar displays 'nand2tetris.github.io/web-ide/compiler'. The page title is 'IAND2Tetris / Jack Compiler / src / *.jack'. Below the title, there are tabs for 'Display.jack', 'Main.jack', 'Player.jack', and 'Walls.jack'. The 'Main.jack' tab is active, showing the following code:

```
4 // Display module handles the following services:
5 // ... rendering the walls with the correct textures and heights
6 // ... rendering the bitmaps
7 // ... rendering text to the screen
8
9 class Display {
10
11     // Bit-masks to simplify rendering
12     static Array patternMask;
13
14     // Display strings
15     // Note that the OS implementation leaks memory, so we have to
16     // pre-initialise the strings
17     static String stepStr, targetStr, text1, text2, text3;
18
19     // Screen location for rendering
20     static int screenLocation;
21
22     // 8 pattern strip counters for rendering
23     // ... cp is pattern segment counter
24     // ... sp is the current pattern segment (0-7)
25     // ... pt is the pattern mask for the top segment
26     // ... pb is the pattern mask for the bottom segment
27     // ... pp is the bit-mask for the bottom segment
28     // ... wp is the pattern word for this pixel
29     // ... wd is the distance word for this pixel
30     static int cp0, cp1, cp2, cp3, cp4, cp5, cp6, cp7;
31     static int sp0, sp1, sp2, sp3, sp4, sp5, sp6, sp7;
```

Below the code, there is a status bar that says 'Compiled successfully'. A red dashed box highlights the status bar, and a red arrow points to it from the right. Another red arrow points to the 'Run' button in the top toolbar.

Desenvolvendo uma aplicação Jack

- **Execute** a aplicação: O código compilado será carregado no emulador de máquina virtual e passa o controle para o emulador de máquina virtual.

← → ↻ nand2tetris.github.io/web-ide/compiler

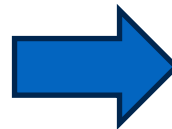
NAND2Tetris / Jack Compiler / src / *.jack

Source + Compile Run

Display.jack Main.jack Player.jack Walls.jack

```
4 // Display module handles the following services:
5 // ... rendering the walls with the correct textures and h
6 // ... rendering the bitmaps
7 // ... rendering text to the screen
8
9 class Display {
10
11     // Bit-masks to simplify rendering
12     static Array patternMask;
13
14     // Display strings
15     // Note that the OS implementation leaks memory, so we
16     // pre-initialise the strings
17     static String stepStr, targetStr, text1, text2, text3;
18
19     // Screen location for rendering
20     static int screenLocation;
```

Clique



NAND2Tetris / VM Emulator

VM Code

```
1 function Main.fibonacci 0
2   push argument 0
3   push constant 2
4   lt
5   if-goto IF_TRUE
6   goto IF_FALSE
7   label IF_TRUE
8   push argument 0
9   return
10  label IF_FALSE
11  push argument 0
12  push constant 2
13  sub
14  call Main.fibonacci 1 // computes fib(n-2)
15  push argument 0
16  push constant 0
17  sub
18  call Main.fibonacci 1 // computes fib(n-1)
19  add
20  // returns fib(n-1) + fib(n-2)
```

Screen

Test Script

```
1 repeat {
2   vmstep;
3 }
```

Enable Keyboard Key: Char code: 0

RAM	Addr	Value	RAM	Addr	Value
SP:	0	256			
LCL:	1	0			
ARG:	2	0			
THIS:	3	0			
THAT:	4	0			
TEMP0:	5	0			
TEMP1:	6	0			
TEMP2:	7	0			
TEMP3:	8	0			
TEMP4:	9	0			
TEMP5:	10	0			
TEMP6:	11	0			

Stack: []

Call-stack: Sys.init

Desenvolvendo uma aplicação Jack

- O emulador de máquina virtual pode então ser usado para executar o código compilado.

Deslize essa barra todo para o lado do fast (direito)

NAND2Tetris / VM Emulator

VM Code

```
1 function Main.fibonacci
2   push argument 0
3   push constant 2
4   lt
5   if-goto IF_TRUE // checks if n<2
6   goto IF_FALSE
7   label IF_TRUE // if n<2, return n
8   push argument 0
9   return
10  label IF_FALSE // if n>=2, returns fib(n-2)+fib(n-1)
11  push argument 0
12  push constant 2
13  sub
14  call Main.fibonacci 1 // computes fib(n-2)
15  push argument 0
16  push constant 1
17  sub
18  call Main.fibonacci 1 // computes fib(n-1)
19  add // returns fib(n-1) + fib(n-2)
```

VM Structures

Stack: []

Call-stack: Sys.init

Screen

Enable Keyboard Key: Char code: 0

RAM	Addr	Value	RAM	Addr	Value
SP:	0	256		256	0
LCL:	1	0		257	0
ARG:	2	0		258	0
THIS:	3	0		259	0
THAT:	4	0		260	0
TEMP0:	5	0		261	0
TEMP1:	6	0		262	0
TEMP2:	7	0		263	0
TEMP3:	8	0		264	0
TEMP4:	9	0		265	0
TEMP5:	10	0		266	0
TEMP6:	11	0		267	0

Test

Test Script

```
1 repeat {
2   vmstep;
3 }
```

Clear

Test

Desenvolvendo uma aplicação Jack

- O emulador de máquina virtual pode então ser usado para executar o código compilado.

Em seguida, clique em run

The screenshot displays the NAND2Tetris VM Emulator interface. The main window is divided into several sections:

- VM Code:** A text editor showing a Fibonacci function in Jack code. The code is as follows:

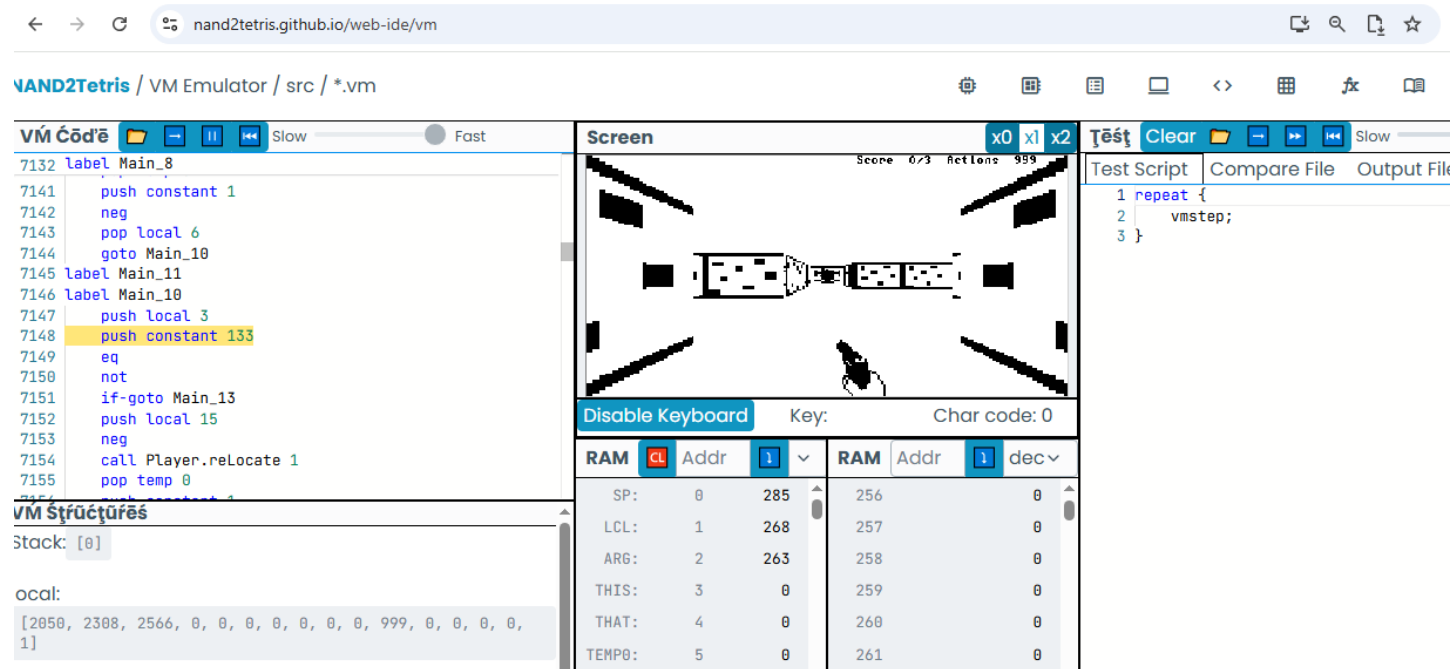
```
1 function Main.fibonacci 0
2   push argument 0
3   push constant 2
4   lt           // checks if n<2
5   if-goto IF_TRUE
6   goto IF_FALSE
7   label IF_TRUE // if n<2, return n
8   push argument 0
9   return
10  label IF_FALSE // if n>=2, returns fib(n-2)+fib(n-1)
11  push argument 0
12  push constant 2
13  sub
14  call Main.fibonacci 1 // computes fib(n-2)
15  push argument 0
16  push constant 1
17  sub
18  call Main.fibonacci 1 // computes fib(n-1)
19  add // returns fib(n-1) + fib(n-2)
```
- Stack:** Shows the current stack state. The stack is empty, and the call stack shows 'Sys.init'.
- Screen:** A window for the emulated screen, currently blank.
- Registers:** A table showing the state of various registers. The 'RAM' section is expanded, showing addresses and values for SP, LCL, ARG, THIS, THAT, and TEMP registers.
- Test:** A section for testing the code, with a 'Test Script' field containing a simple repeat loop.

The 'RAM' section of the registers table is as follows:

Register	Addr	Value
SP	0	256
LCL	1	0
ARG	2	0
THIS	3	0
THAT	4	0
TEMP0	5	0
TEMP1	6	0
TEMP2	7	0
TEMP3	8	0
TEMP4	9	0
TEMP5	10	0
TEMP6	11	0

Desenvolvendo uma aplicação Jack

- O emulador de máquina virtual pode então ser usado para executar o código compilado.



Desenvolvendo uma aplicação Jack

- O emulador de máquina virtual pode então ser usado para executar o código compilado.

Se for utilizar o teclado para interagir, clique aqui para habilitá-lo

VM Emulator / src / *.vm

VM Code

```
7132 label Main_8
7141   push constant 1
7142   neg
7143   pop local 6
7144   goto Main_10
7145 label Main_11
7146 label Main_10
7147   push local 3
7148   push constant 133
7149   eq
7150   not
7151   if-goto Main_13
7152   push local 15
7153   neg
7154   call Player.reLocate 1
7155   pop temp 0
```

VM Structures

Stack: [0]

local:

```
[2050, 2308, 2566, 0, 0, 0, 0, 0, 0, 0, 999, 0, 0, 0, 0, 1]
```

Screen

Score: 0/3 Actions: 999

Disable Keyboard Key: Char code: 0

RAM	Addr	dec
SP:	0	285
LCL:	1	268
ARG:	2	263
THIS:	3	0
THAT:	4	0
TEMP0:	5	0

Test

Test Script

```
1 repeat {
2   vmstep;
3 }
```

Executando uma aplicação Jack

File View Run Help

Animate: Program flow View: Scr... Format: D...

Program

```
0 push constant 10
1 pop local 0
2 push constant 21
3 push constant 22
4 pop argument 2
5 pop argument 1
6 push constant 36
7 pop this 6
8 push constant 42
9 push constant 45
10 pop that 5
11 pop that 2
12 push constant 510
13 pop temp 6
14 push local 0
```

Static

0	0
1	0
2	0
3	0
4	0

Local

0	10
1	0
2	0
3	0
4	0

Argument

0	0
1	21
2	22
3	0
4	0

This

2	0
3	0
4	0
5	0
6	36

That

2	0
3	0

Temp

0	0
---	---

Stack

42
45

Call Stack

--

Global Stack

256	42
257	45
258	0
259	0
260	0
261	0
262	0
263	0
264	0
265	0
266	0
267	0
268	0
269	0
270	0

RAM

SP:	0	258
LCL:	1	300
ARG:	2	400
THIS:	3	3000
THAT:	4	3010
Temp0:	5	0
Temp1:	6	0
Temp2:	7	0
Temp3:	8	0
Temp4:	9	0
Temp5:	10	0
Temp6:	11	0
Temp7:	12	0
R13:	13	0
R14:	14	0

Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Lidando com as saídas: texto

- Aplicações textuais:
 - **Tela:** 23 linhas com 64 caracteres, p&b
 - **Fonte:** definida pelo SO Jack
 - **Saída:** classe Output do SO Jack

```
class Output {  
    function void moveCursor(int i, int j)  
    function void printChar(char c)  
    function void printString(String s)  
    function void printInt(int i)  
    function void println()  
    function void backSpace()  
}
```

classe do SO, para manipular
a saída textual

```
Enter the students data, ending with 'Q':  
Name: DAN  
Grade: 90  
Name: PAUL  
Grade: 80  
Name: LISA  
Grade: 100  
Name: ANN  
Grade: 90  
Name: Q  
The grades average is 90  
The student with the highest grade is LISA
```

Usage example:

```
// (Note: the ASCII code of the character a is 97)  
// Prints 97, then prints the character a, twice  
do Output.printInt(97);  
do Output.printChar(97);  
do Output.printString("a");  
...
```

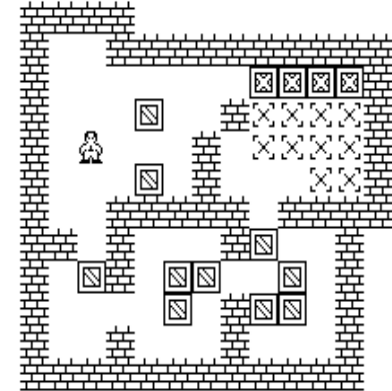
Lidando com as saídas: gráfico

- Aplicações gráficas:
 - **Tela:** 256 linhas com 512 pixels, p&b
 - **Saída:** classe Screen do SO Jack
 - também é possível construir uma classe Screen customizada

```
Class Screen {  
    function void clearScreen()  
    function void setColor(boolean b)  
    function void drawPixel(int x, int y)  
    function void drawLine(int x1, int y1, int x2, int y2)  
    function void drawRectangle(int x1, int y1, int x2, int y2)  
    function void drawCircle(int x, int y, int r)  
}
```

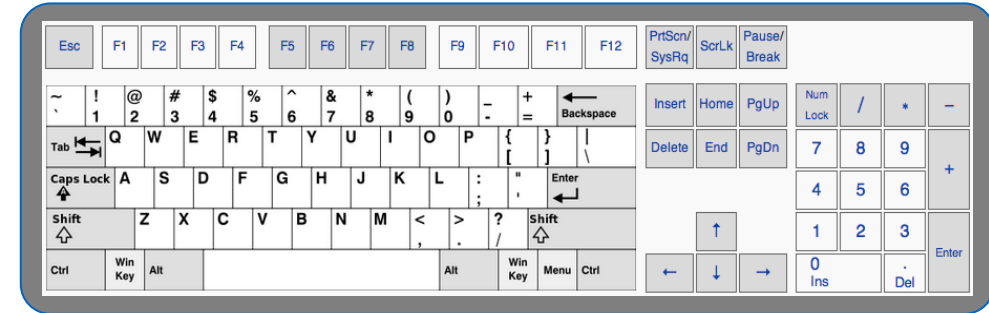
classe do SO, para manipular a saída gráfica

```
*****  
*** SOKOBAN ***  
*****  
  
Status:  
  Level:  1  
  Pushes: 38  
  
Commands:  
  [1-5] Level  
  [U]ndo  
  [R]estart  
  [S]olution  
  [Q]uit
```



Lidando com as entradas

- Dispositivo de entrada:
 - Teclado padrão
 - Gerenciando as entradas: utilizar a classe do SO, Keyboard



```
Class Keyboard {  
    function char keyPressed()  
    function char readChar()  
    function String readLine(String message)  
    function int readInt(String message)  
}
```

classe do SO, para gerenciar as entradas do teclado

Conjunto de caracteres do Hack

key	code
(space)	32
!	33
“	34
#	35
\$	36
%	37
&	38
‘	39
(	40
)	41
*	42
+	43
,	44
-	45
.	46
/	47

key	code
0	48
1	49
...	...
9	57

:	58
;	59
<	60
=	61
>	62
?	63
@	64

key	code
A	65
B	66
C	...
...	...
Z	90

[	91
/	92
]	93
^	94
_	95
`	96

key	code
a	97
b	98
c	99
...	...
z	122

{	123
	124
}	125
~	126

key	code
newline	128
backspace	129
left arrow	130
up arrow	131
right arrow	132
down arrow	133
home	134
end	135
Page up	136
Page down	137
insert	138
delete	139
esc	140
f1	141
...	...
f12	152

a função `Keyboard.keypress()` retorna o **código** da tecla pressionada no momento de sua chamada, ou 0 caso não haja nenhuma

0 Sistema Operacional Jack: Math

```
class Math {  
    function void init()  
    function int abs(int x)  
    function int multiply(int x, int y)  
    function int divide(int x, int y)  
    function int min(int x, int y)  
    function int max(int x, int y)  
    function int sqrt(int x)  
}
```



0 Sistema Operacional Jack: String

```
Class String {  
    constructor String new(int maxLength)  
    method void    dispose()  
    method int     length()  
    method char    charAt(int j)  
    method void    setCharAt(int j, char c)  
    method String  appendChar(char c)  
    method void    eraseLastChar()  
    method int     intValue()  
    method void    setInt(int j)  
    function char  backSpace()  
    function char  doubleQuote()  
    function char  newLine()  
}
```

0 Sistema Operacional Jack: Array

```
Class Array {  
    function Array new(int size)  
    method void dispose()  
}
```

0 Sistema Operacional Jack: Memory

```
class Memory {  
    function int peek(int address)  
    function void poke(int address, int value)  
    function Array alloc(int size)  
    function void deAlloc(Array o)  
}
```



0 Sistema Operacional Jack: Sys

```
Class Sys {  
    function void halt():  
    function void error(int errorCode)  
    function void wait(int duration)  
}
```



Programas de exemplo Jack

- **Square:** uma aplicação OO simples, interativa, com múltiplas classes
- **Pong:** uma aplicação OO completa, interativa, com múltiplas classes
- **Average:** ilustra a manipulação simples de arrays
- **ComplexArrays:** ilustra várias manipulações de arrays, incluindo arrays de duas dimensões
- **ConvertToBin:** ilustra operações algébricas e o acesso à memória utilizando as funções `peek` e `poke`

código disponível em: nand2tetris/projects/11



UFRPE

Boas Práticas

- **Gerais:**

- Veja alguns programas Jack de exemplo (“cool stuff” (coisas legais) em www.nand2tetris.org)
- Brinque com os programas fornecidos e revise os códigos (por exemplo, Square e Pong)
- Entenda as limitações da interface com o usuário do Jack
 - A API do sistema operacional está em nand2tetris / projects / 09 (Jack OS API.pdf)
- Planeje sua aplicação com cuidado (design OO e estratégia de teste)
- Implemente, teste e ... divirta-se!



UFRPE

Boas Práticas

- **Técnicas:**

- **Escrita:** Grave / edite seus arquivos das classes Jack em um editor de texto padrão;
- **Otimizando:** deixe para o final
- **Documentando:** use a prática padrão
- **Compilando:** use o **JackCompiler** fornecido (de preferência o online)
- **Executando:** carregue o diretório do aplicativo (que agora contém os arquivos .vm compilados) no emulador de VM fornecido e execute o código
 - (lembre-se dos controles de velocidade e animação do emulador).



Linguagem de alto nível: como será abordado?

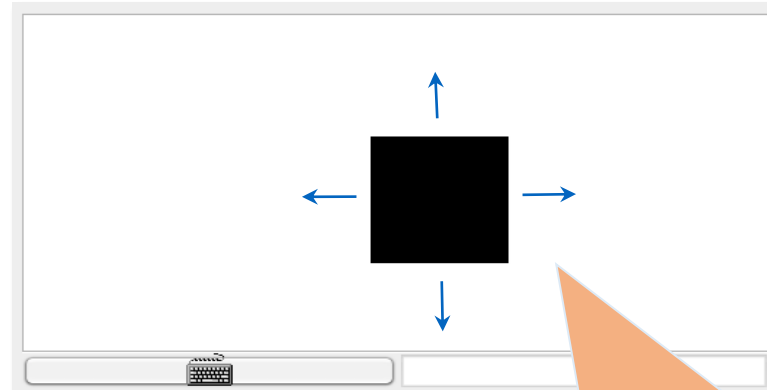
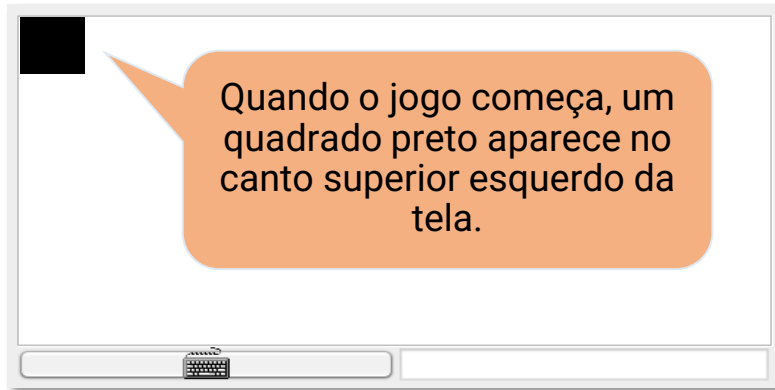
- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Objetivos

- O código da aplicação `Square` ilustra:
 - projeto orientado à objetos
 - uma aplicação interativa típica
 - Manipulação de entradas e saídas
 - Utilização do SO



Jogo *Square Dance*

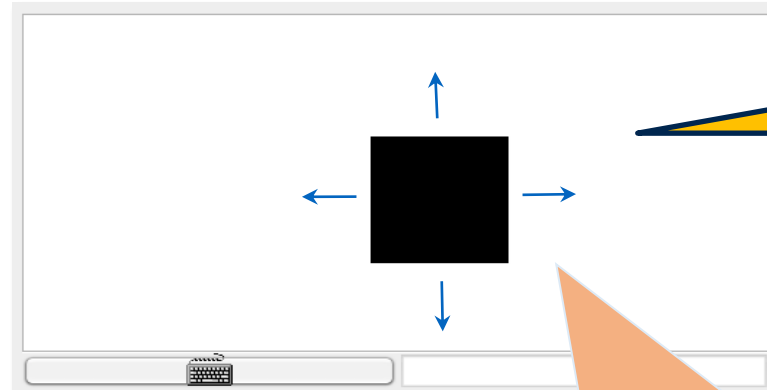
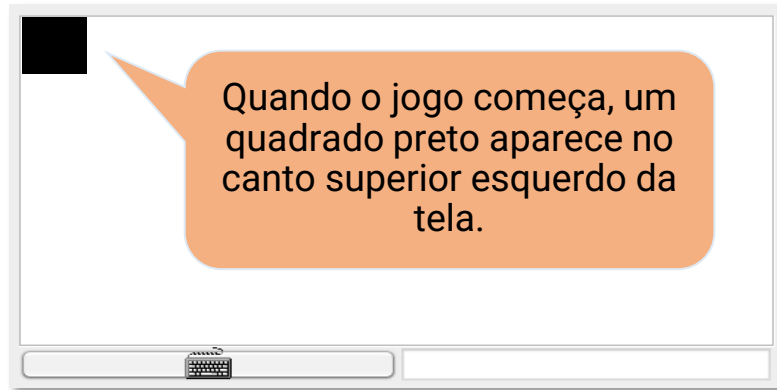


Se o quadrado atingir uma parede, ele permanece lá até que alguma ação do usuário o libere

- **Instruções do jogo:**

- up arrow: the square starts moving up, until another key is pressed
- down arrow: the square starts moving down, until another key is pressed
- left arrow: the square starts moving left, until another key is pressed
- right arrow: the square starts moving right, until another key is pressed
- x key: the square's size increases a little (2 pixels)
- z key: the square's size decreases a little (2 pixels)
- q: game over.

Jogo *Square Dance*



É um código excelente para a gente ver a manipulação do teclado e de pixels

- **Instruções do jogo:**

- up arrow: the square starts moving up, until another key is pressed
- down arrow: the square starts moving down, until another key is pressed
- left arrow: the square starts moving left, until another key is pressed
- right arrow: the square starts moving right, until another key is pressed
- x key: the square's size increases a little (2 pixels)
- z key: the square's size decreases a little (2 pixels)
- q: game over.

Projeto da aplicação

- Três classes Jack:
 - **Square:** representa o objeto quadrado gráfico
 - **SquareGame:**
 - cria um quadrado
 - em seguida, captura as entradas do usuário e move o quadrado de acordo (em um loop).
 - **Main:** Cria um SquareGame e inicia o jogo.

API da classe *Square*

Square API

```
/** Implements a graphical square. */
class Square {

    /** Constructs a new square with a given location and size */
    constructor Square new(int Ax, int Ay, int Asize)

    /** Disposes this square */
    method void dispose()

    /** Draws this square on the screen */
    method void draw()

    /** Erases this square from the screen */
    method void erase()

    /** Increments this square's size by 2 pixels */
    method void incSize()

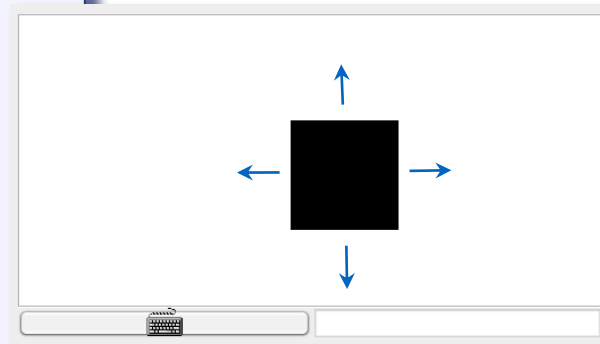
    /** Decrements this square's size by 2 pixels */
    method void decSize()

    /** Moves this square up by 2 pixels */
    method void moveUp()

    /** Moves this square down by 2 pixels */
    method void moveDown()

    /** Moves this square left by 2 pixels */
    method void moveLeft()

    /** Moves this square right by 2 pixels */
    method void moveRight()
}
```



Classe Square

Square.jack

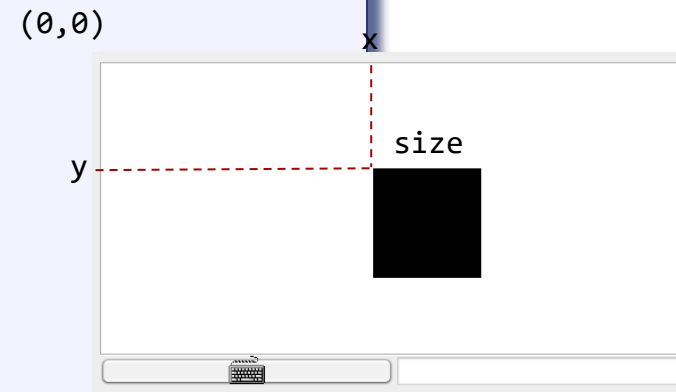
```
/** Implements a graphical square. */
class Square {

    field int x, y; // screen location of the square's top-left corner
    field int size; // length of this square, in pixels

    /** Constructs a new square with a given location and size. */
    constructor Square new(int Ax, int Ay, int Asize) {
        let x = Ax;
        let y = Ay;
        let size = Asize;
        do draw();
        return this;
    }

    /** Disposes this square. */
    method void dispose() {
        do Memory.deAlloc(this);
        return;
    }

    ...
}
```



Classe Square

Square.jack

```
/** Implements a graphical square. */
class Square {

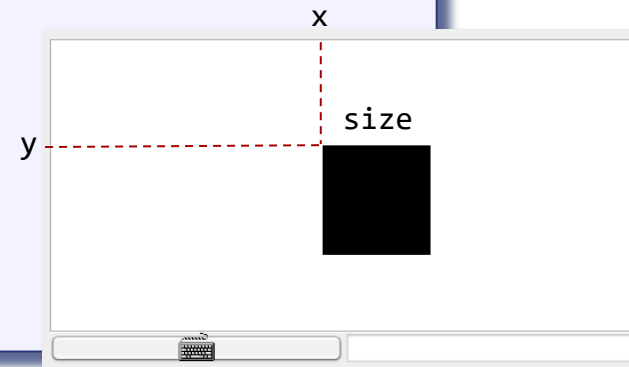
    field int x, y; // screen location of the square's top-left corner
    field int size; // length of this square, in pixels

    ...

    /** Draws the square on the screen. */
    method void draw() {
        do Screen.setColor(true);
        do Screen.drawRectangle(x, y, x + size, y + size);
        return;
    }

    /** Erases the square from the screen. */
    method void erase() {
        do Screen.setColor(false);
        do Screen.drawRectangle(x, y, x + size, y + size);
        return;
    }

    ...
}
```



Classe Square

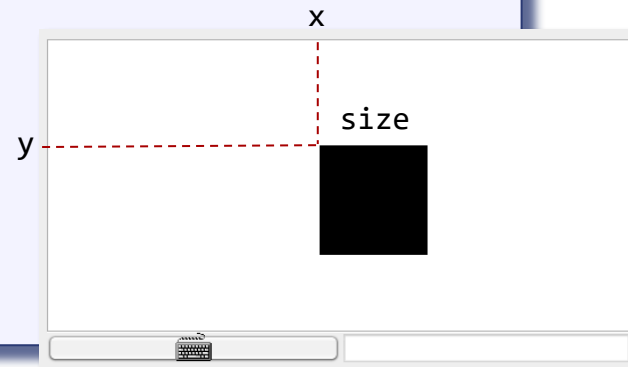
Square.jack

```
/** Implements a graphical square. */
class Square {

    field int x, y; // screen location of the square's top-left corner
    field int size; // length of this square, in pixels
    ...

    /** Increments the square size by 2 pixels. */
    method void incSize() {
        if (((y + size) < 254) & ((x + size) < 510)) {
            do erase();
            let size = size + 2;
            do draw();
        }
        return;
    }

    /** Decrements the square size by 2 pixels. */
    method void decSize() {
        if (size > 2) {
            do erase();
            let size = size - 2;
            do draw();
        }
        return;
    }
    ...
}
```



Classe Square

Square.jack

```
/** Implements a graphical square. */
class Square {

    field int x, y; // screen location of the square's top-left corner
    field int size; // length of this square, in pixels
    ...

    /** Moves the square up by 2 pixels. */
    method void moveUp() {
        if (y > 1) {
            do Screen.setColor(false);
            do Screen.drawRect(x, (y + size) - 1, x + size, y + size);
            let y = y - 2;
            do Screen.setColor(true);
            do Screen.drawRect(x, y, x + size, y + 1);
        }
        return;
    }

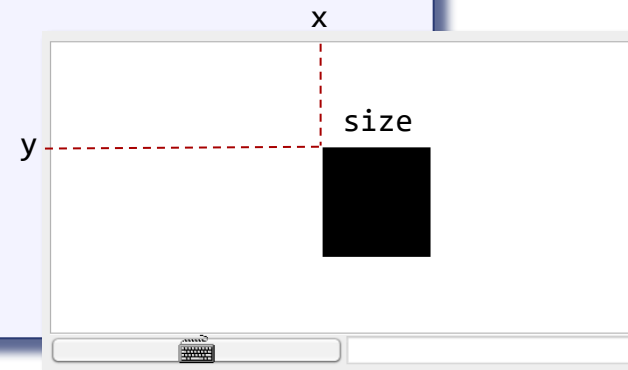
    method void moveDown() { // similar }

    method void moveLeft() { // similar }

    method void moveRight() { // similar }

    ...

} // class Square
```



Classe *SquareGame*

SquareGame.jack

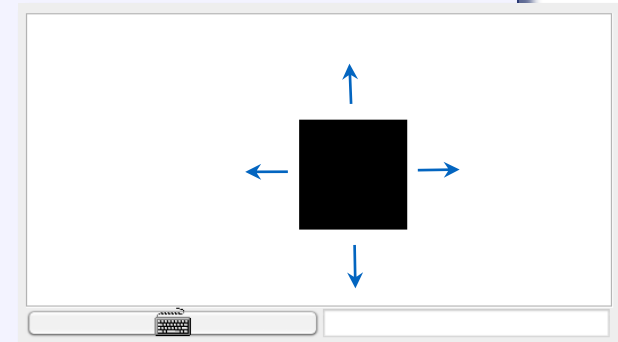
```
/** Implements a square game. */
class SquareGame {

    field Square square; // the square of this game
    field int direction; // the square's current direction:
                        // 0=none, 1=up, 2=down, 3=left, 4=right

    /** Constructs a new Square Game. */
    constructor SquareGame new() {
        let square = Square.new(0, 0, 30);
        let direction = 0;
        return this;
    }

    /** Disposes this game. */
    method void dispose() {
        do square.dispose();
        do Memory.deAlloc(this);
        return;
    }

    ...
}
```



Classe *SquareGame*

SquareGame.jack

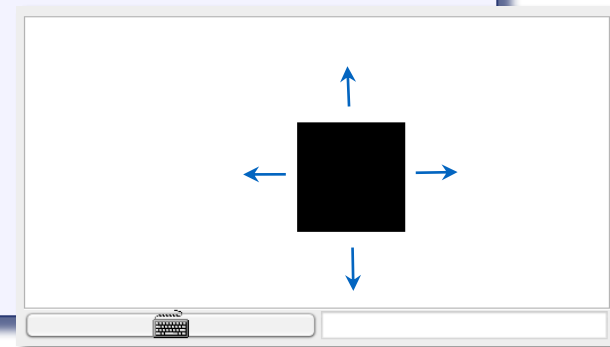
```
/** Implements a square game. */
class SquareGame {

    field Square square; // the square of this game
    field int direction; // the square's current direction:
                        // 0=none, 1=up, 2=down, 3=left, 4=right

    /** Moves the square in the current direction. */
    method void moveSquare() {
        if (direction = 1) { do square.moveUp(); }
        if (direction = 2) { do square.moveDown(); }
        if (direction = 3) { do square.moveLeft(); }
        if (direction = 4) { do square.moveRight(); }

        do Sys.wait(5); // delays the next movement
        return;
    }

    ...
}
```



Classe *SquareGame*

exemplo típico de um código que manipula os eventos do teclado em aplicações Jack interativas.

SquareGame.jack

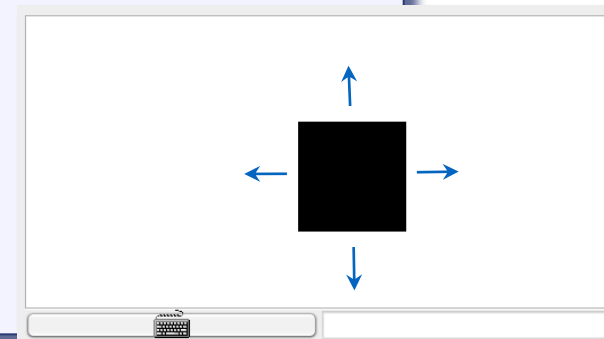
```
/** Implements a square game. */
class SquareGame {

    field Square square; // the square of this game
    field int direction; // the square's current direction:
                        // 0=none, 1=up, 2=down, 3=left, 4=right

    /** Runs the game: handles the user's inputs and moves the square accordingly */
    method void run() {
        var char key; // the key currently pressed by the user
        var boolean exit;
        let exit = false;

        while (~exit) {
            // waits for a key to be pressed
            while (key = 0) {
                let key = Keyboard.keyPressed();
                do moveSquare();
            }
            if (key = 81) { let exit = true; } // q key
            if (key = 90) { do square.decSize(); } // z key
            if (key = 88) { do square.incSize(); } // x key
            if (key = 131) { let direction = 1; } // up arrow
            if (key = 133) { let direction = 2; } // down arrow
            if (key = 130) { let direction = 3; } // left arrow
            if (key = 132) { let direction = 4; } // right arrow

            // waits for the key to be released
            while (~(key = 0)) {
                let key = Keyboard.keyPressed();
                do moveSquare();
            }
        } // while
        return;
    }
} // SquareGame class
```

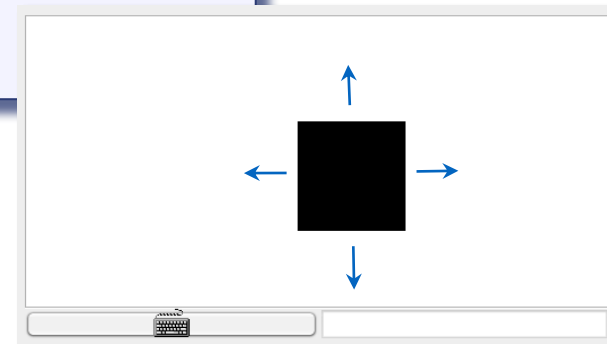


Classe *Main*

Main.jack

```
/**
 * Main class of the Square Dance game.
 */
class Main {

    /** Initializes a new game and starts it. */
    function void main() {
        var SquareGame game;
        let game = SquareGame.new();
        do game.run();
        do game.dispose();
        return;
    }
}
```



Recapitulando...

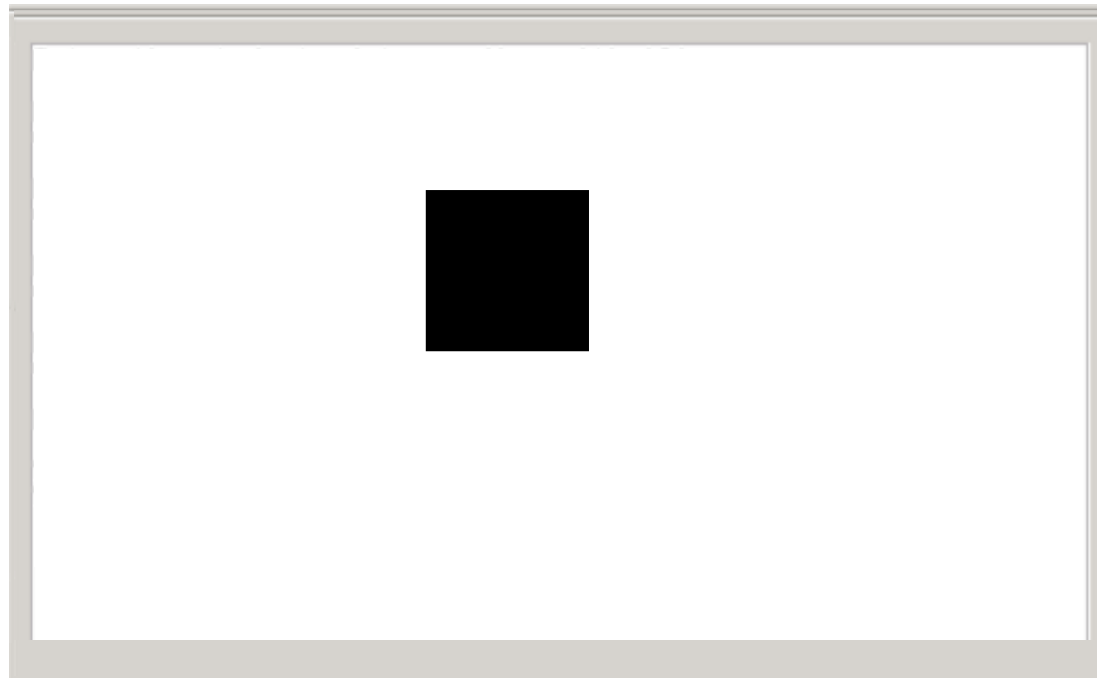
- **Questões importantes:**
 - compreender o estilo de interface com o usuário
 - conhecer o SO Jack
 - projeto OO
 - estratégia de testes



Linguagem de alto nível: como será abordado?

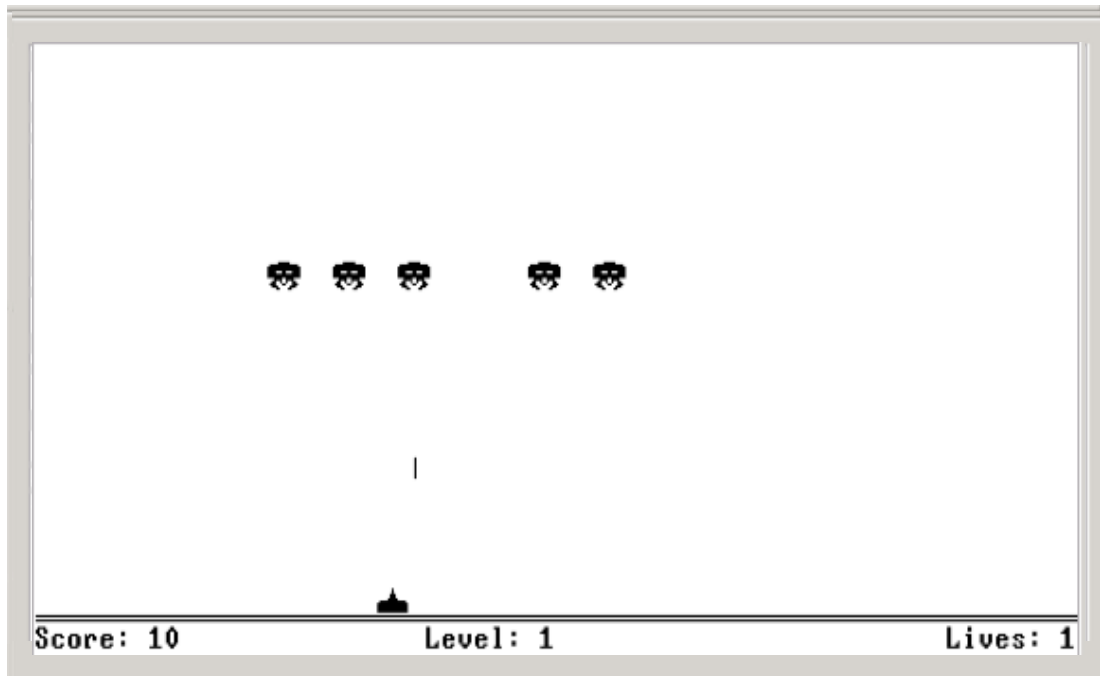
- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

Square dance



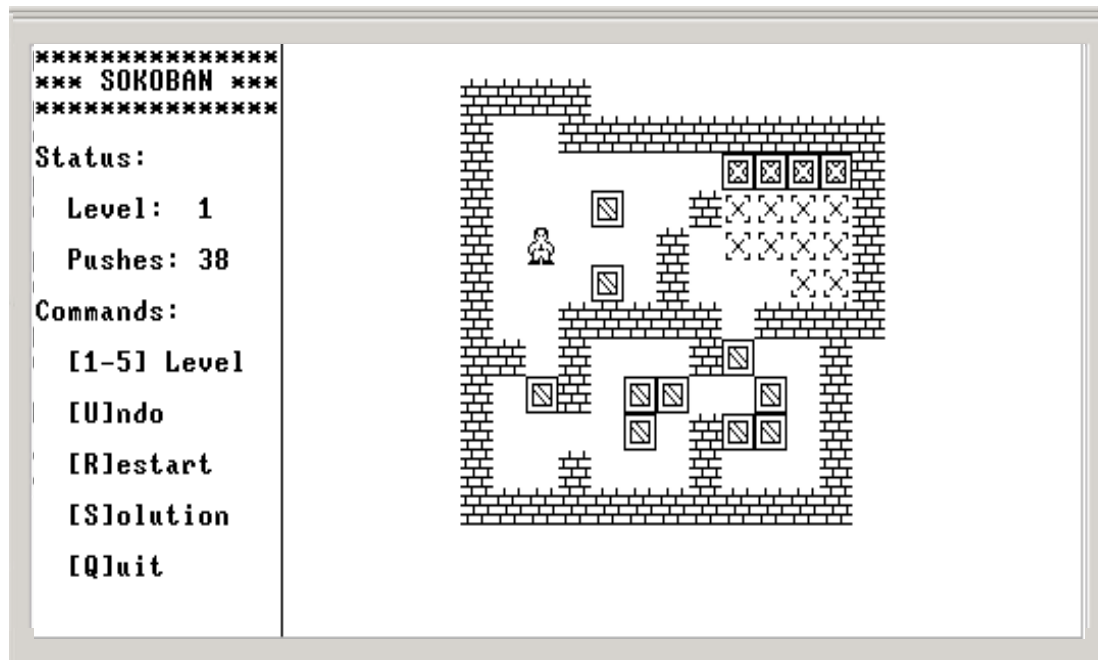
Space Invaders (by Ran Navok)

- Interfaces mais complexas



Sokoban (by Golan Parashi)

- Interfaces mais complexas



Lidando com os *sprites*

Conceito de jogos de computador

- ***Sprite*:**

- um *bitmap* de bidimensional, tipicamente integrado a um cenário maior

- **Desafios:**

- desenhar *sprites* rapidamente
- criar animações suaves

- **Soluções:**

- Utilizar a biblioteca gráfica padrão do SO (nem sempre é eficiente)
- Utilizar suas próprias funções gráficas

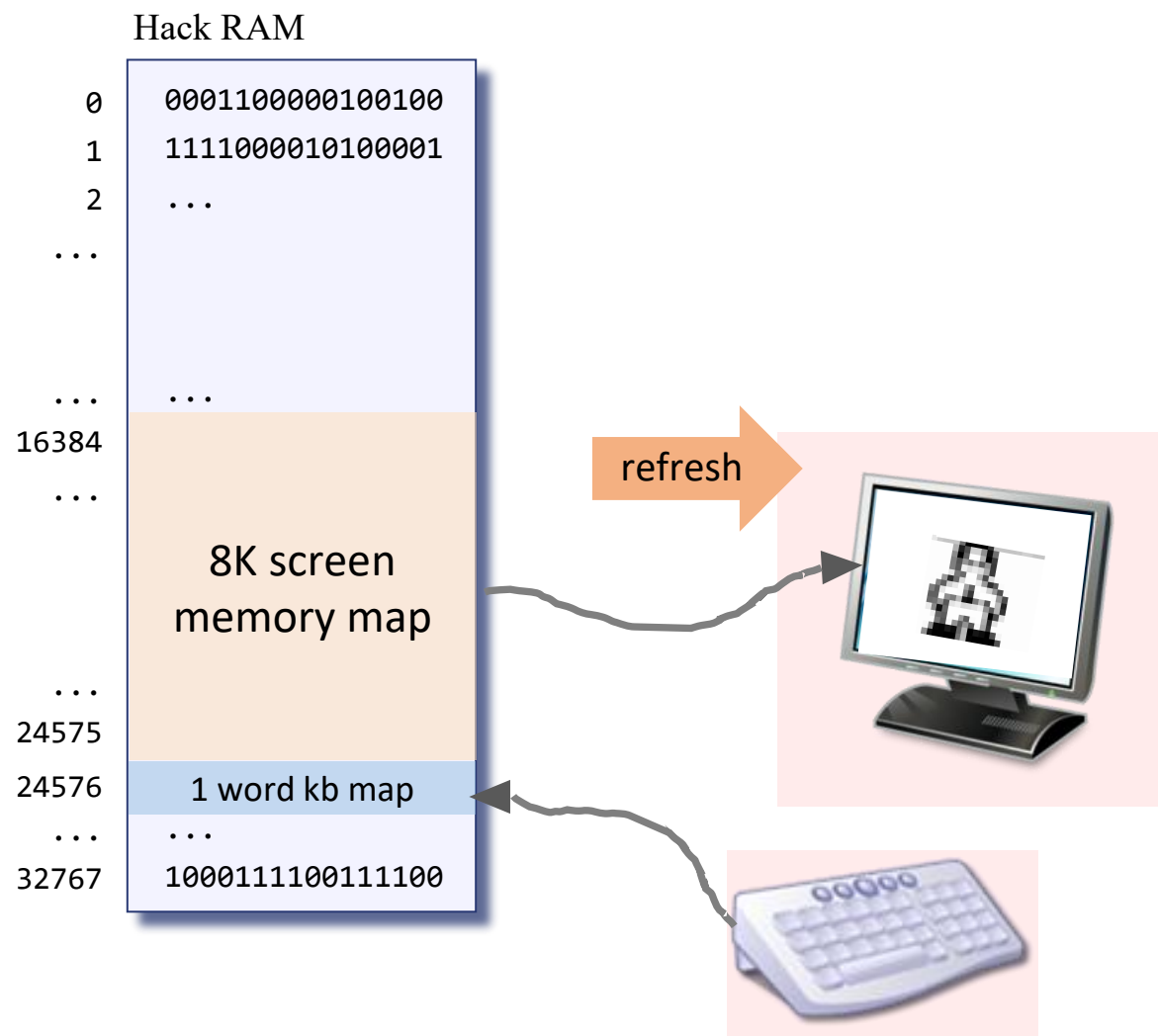


Hack I/O

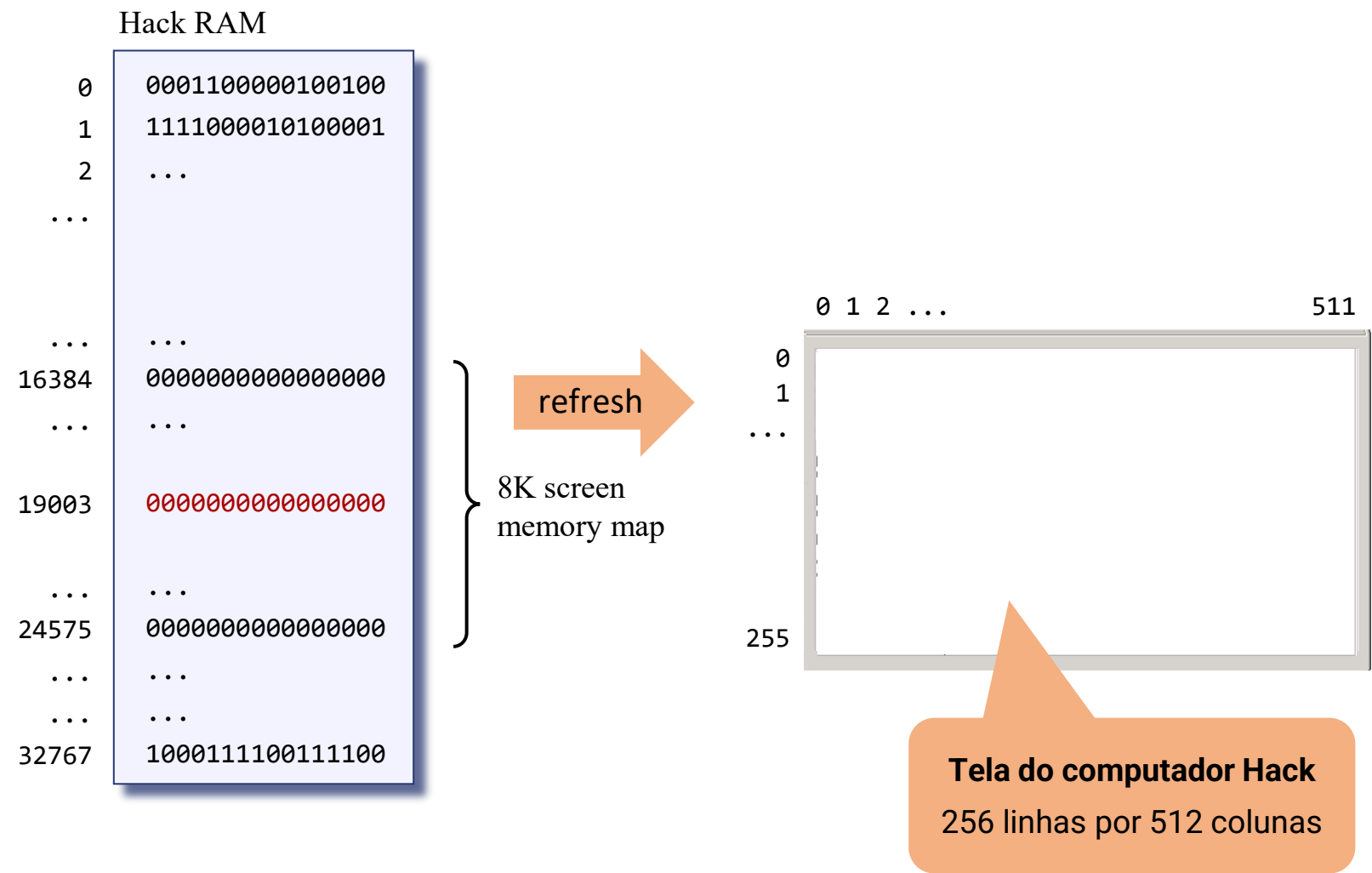
Hack RAM

[illegible]

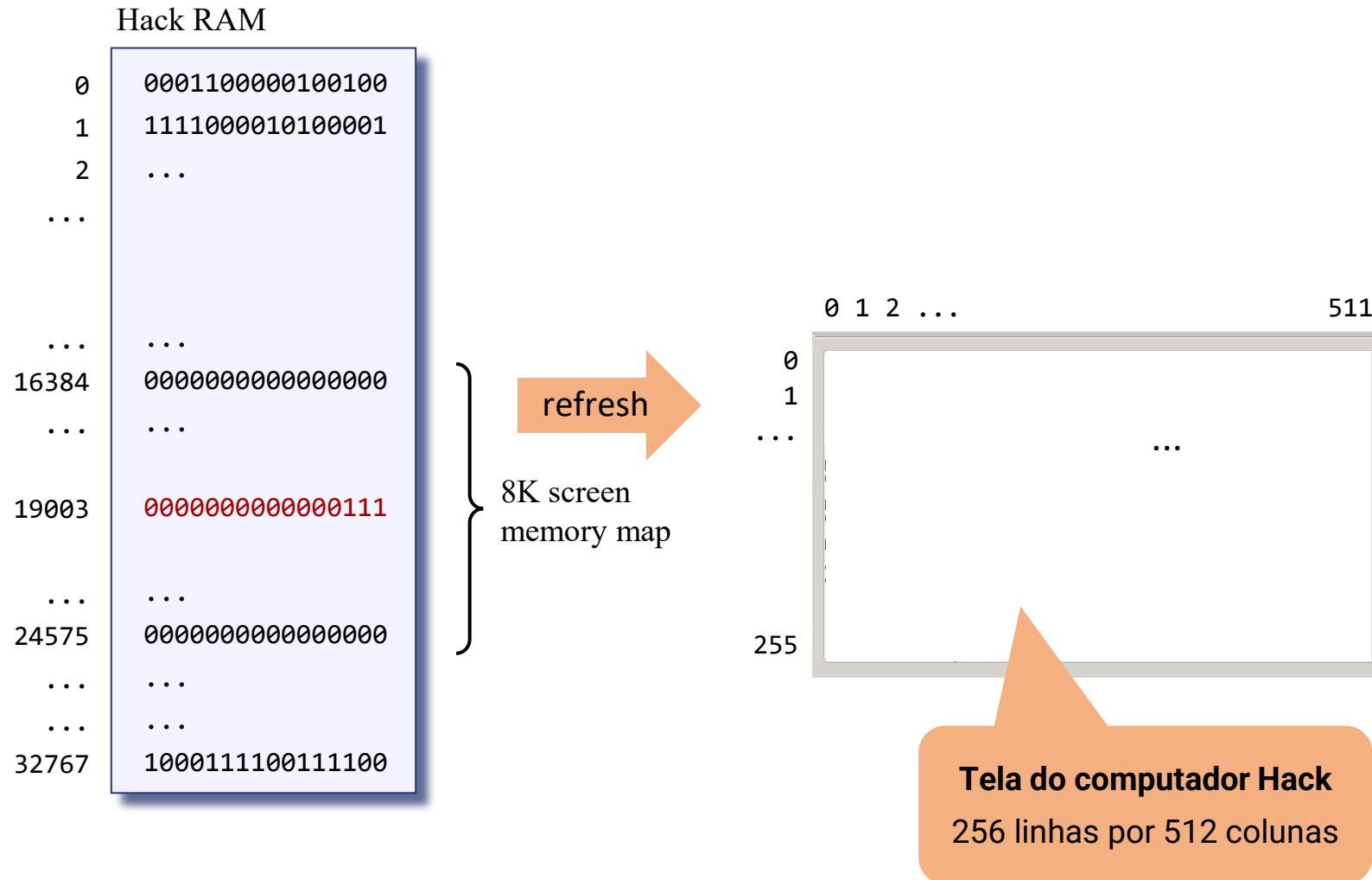
Hack I/O



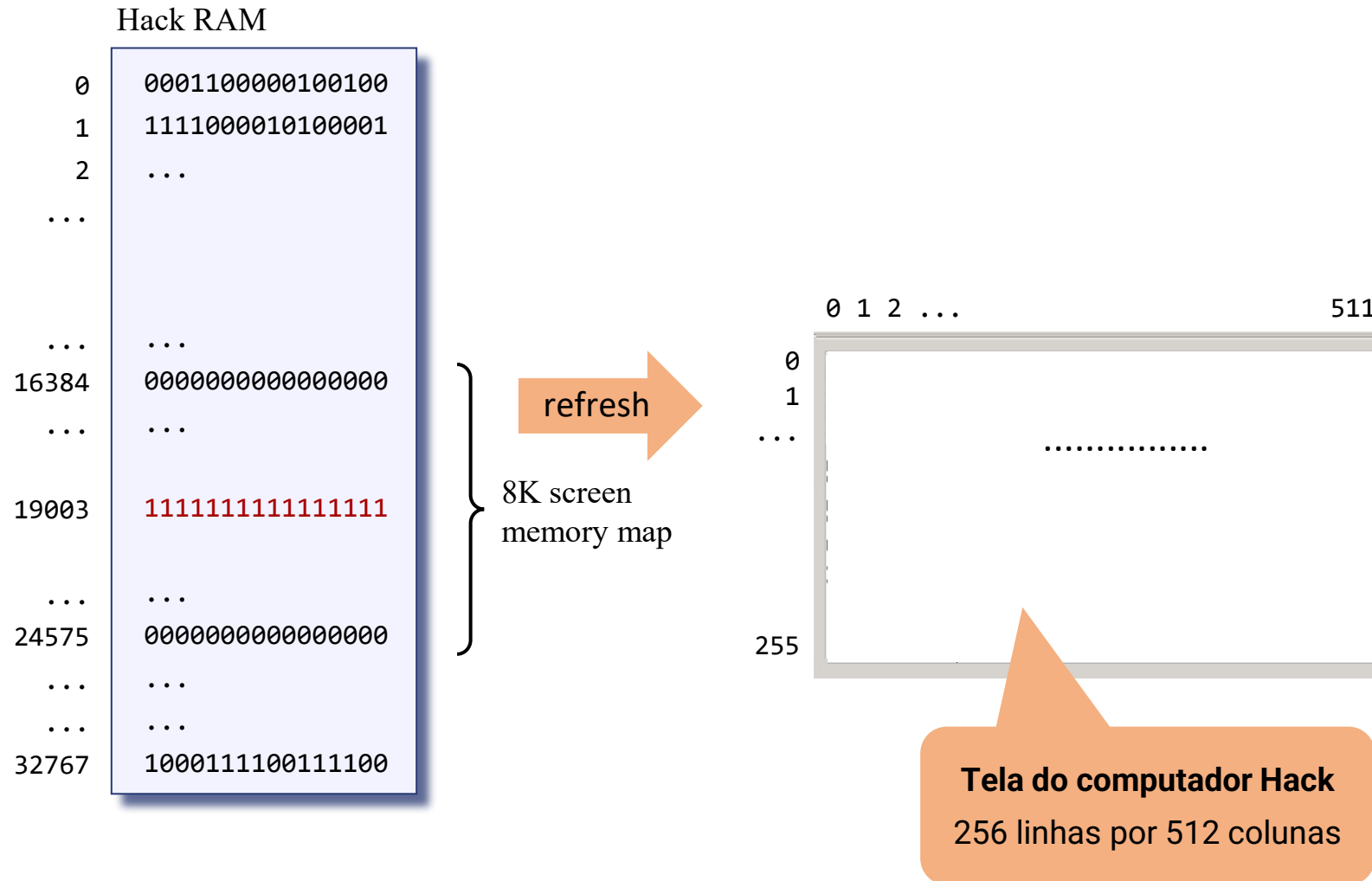
Mapeamento em memória



Mapeamento em memória



Mapeamento em memória



Acessando a memória: leitura / escrita

Hack RAM

0	0001100000100100
1	1111000010100001
2	...
...	
...	...
16384	0000000000000000
...	...
19003	0000000000000111
...	...
24575	0000000000000000
...	...
...	...
32767	1000111100111100

A classe **Memory** da API do SO:

- function int `peek`(int address)
- function void `poke`(int address, int value)
- ...

Jack code

```
let x = Memory.peek(19003)  
// x will be set to 7
```

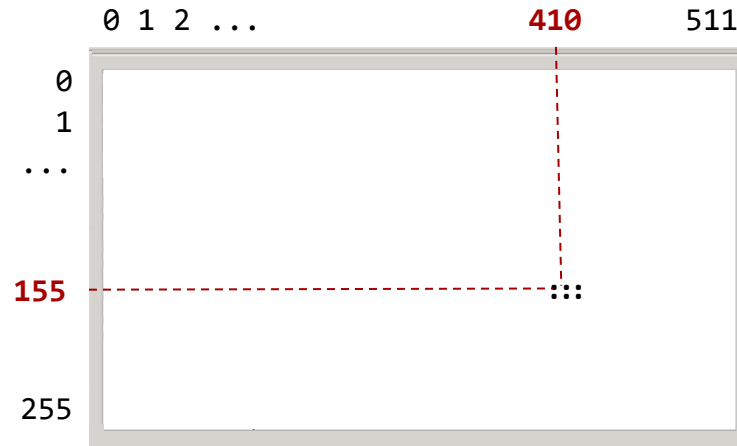
```
do Memory.poke(19003, -1)  
// RAM[19003] will be set to  
// 1111111111111111
```



Desenhando pixels

Jack code

```
// draws the image
do Screen.drawPixel(410,155);
do Screen.drawPixel(411,155);
do Screen.drawPixel(412,155);
do Screen.drawPixel(410,156);
do Screen.drawPixel(411,156);
do Screen.drawPixel(412,156);
```



A classe **Screen** da API do SO:

- function void **setColor**(boolean b)
- function void **drawPixel**(int x, int y)
- function void **drawLine**(int x1, int y1, int x2, int y2)
- function void **drawRectangle**(int x1, int y1, int x2, int y2)
- function void **drawCircle**(int x, int y, int r):

Desenhando pixels

Jack code

// draws the image

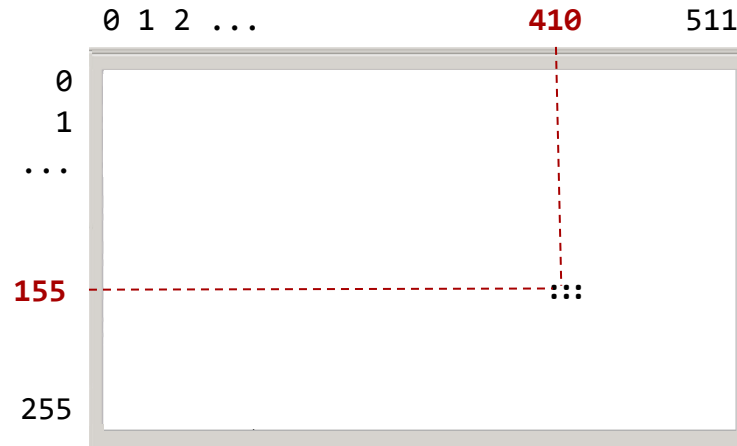
```
do Screen.drawPixel(410,155);  
do Screen.drawPixel(411,155);  
do Screen.drawPixel(412,155);  
do Screen.drawPixel(410,156);  
do Screen.drawPixel(411,156);  
do Screen.drawPixel(412,156);
```

// draws the image

```
do Screen.drawLine(410,155,412,155);  
do Screen.drawLine(410,156,412,156);
```

// draws the image

```
do Screen.drawRectangle(410,155,412,156);
```



Como desenhar os *sprites*?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1																
2																
3																
4																
5																
6																
7																
8																
9																
10																
11																
12																
13																
14																
15																
16																

Image drawing code:

```
// Draws the top row
do Screen.drawPixel(6,1);
do Screen.drawPixel(7,1);
...
do Screen.drawPixel(12,1);
...
// Draws the bottom row
do Screen.drawPixel(3,16);
...
do Screen.drawPixel(15,16);
```

- **Eficiência:**

- 75 operações de desenho de pixels

Implementação da função `drawPixel(x,y)` no S0:

```
// sets pixel (x,y) to black / white:
address = 32 * y + x / 16
value = Memory.peek[16384 + address]
set the (x % 16)th bit of value to 0 or 1
do Memory.poke(address,value)
```

Como desenhar os *sprites*?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1																
2																
3																
4																
5																
6																
7																
8																
9																
10																
11																
12																
13																
14																
15																
16																

00001111111100000

0001100000110000

0001001010010000

...

...

0111111011111100



UFRPE

Como desenhar os *sprites*?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1																
2																
3																
4																
5																
6																
7																
8																
9																
10																
11																
12																
13																
14																
15																
16																

00001111111100000 = 4064

0001100000110000 = 6192

0001001010010000 = 4752

...

...

0111111011111100 = 32508



UFRPE

Como desenhar os *sprites*?

Image drawing code:

```
// Draws the sprite  
do Memory.poke(addr0, 4064);  
do Memory.poke(addr1, 6192);  
do Memory.poke(addr2, 4752);  
...  
do Memory.poke(addr15, 32508);
```

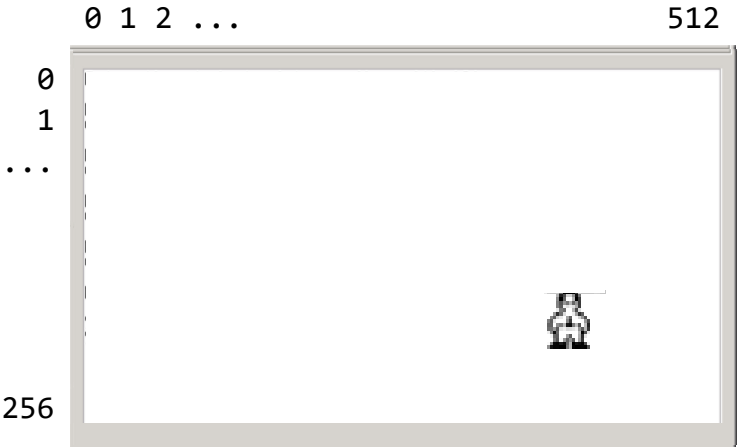
- **Eficiência:**

- 16 operações de escrita na memória

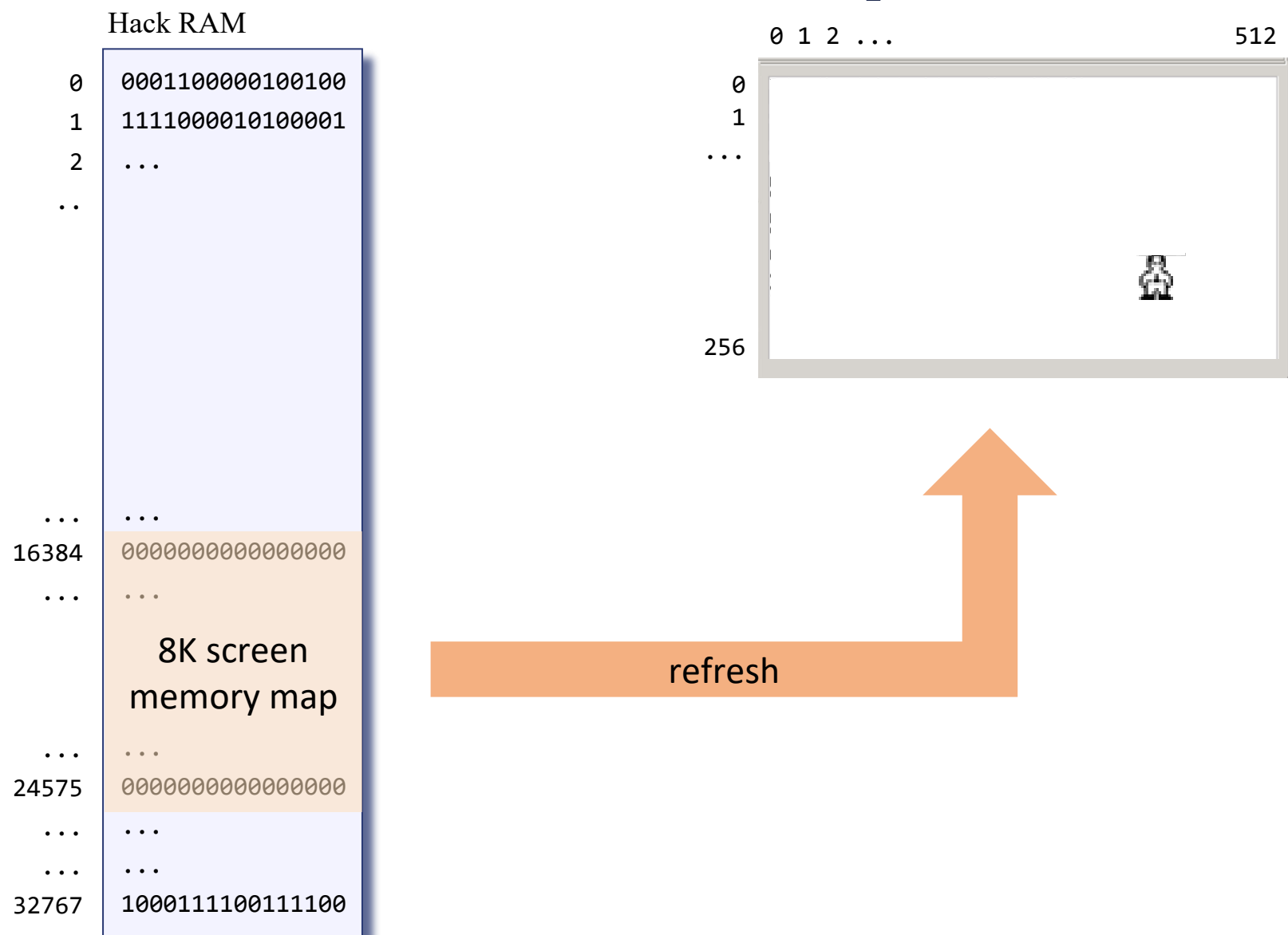
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
1																	4064
2																	6192
3																	4752
4																	...
5																	
6																	
7																	
8																	
9																	
10																	
11																	
12																	
13																	
14																	
15																	...
16																	32508

Como desenhar os *sprites*?

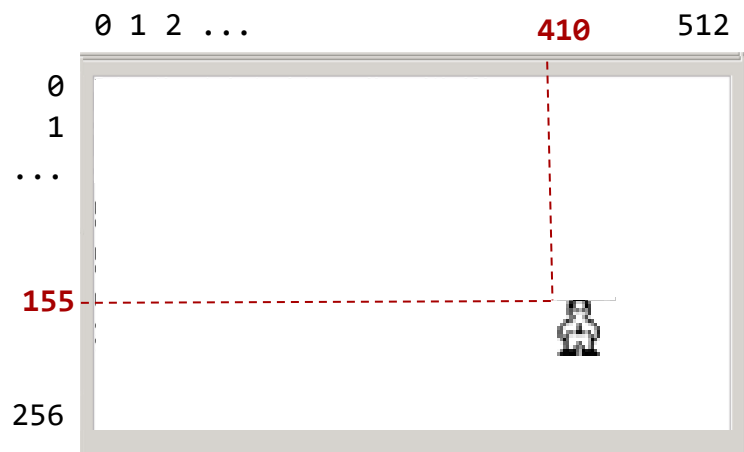
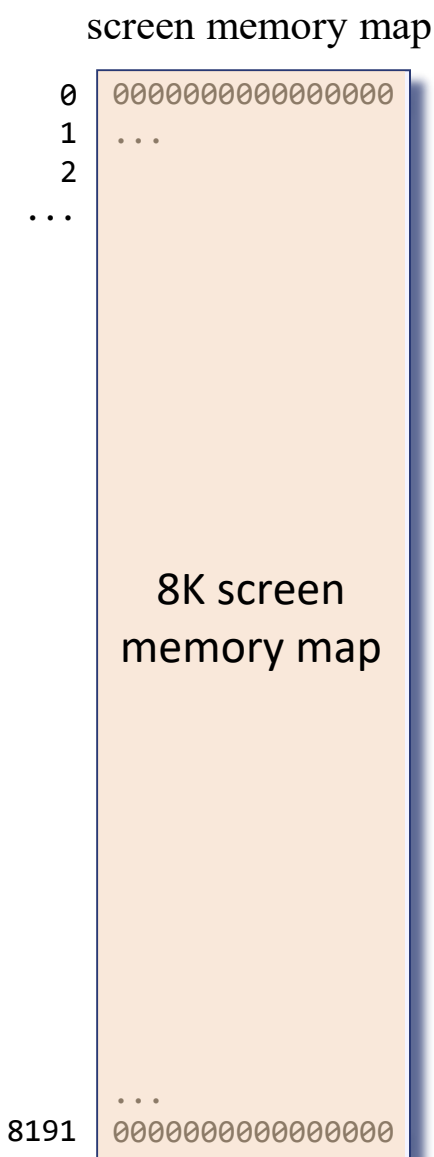
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1																
2																
3																
4																
5																
6																
7																
8																
9																
10																
11																
12																
13																
14																
15																
16																



Como desenhar os *sprites*?



Como desenhar os *sprites*?



Como desenhar os *sprites*?

screen memory map

0	0000000000000000	row 0
...	...	
31	0000000000000000	
32	0000000000000000	row 1
...	...	
63	0000000000000000	
...	...	
...	...	
addr	0000011111111000	row 155
...	...	
addr + 32	0000110000011000	row 156
...	...	
addr + 64	0000100101001000	row 157
...	...	
...	...	
addr + 480	0111111011111100	row 170
...	...	
...	...	
8191	0000000000000000	row 256
...	...	
	0000000000000000	

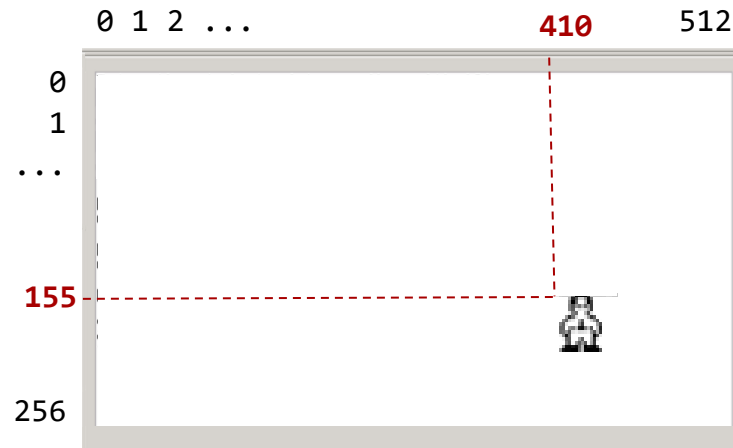


Image drawing code:

```
// Draws the sprite
do Memory.poke(addr + 0, 4064);
do Memory.poke(addr + 32, 6192);
do Memory.poke(addr + 64, 4752);
...
do Memory.poke(addr + 480, 32508);
```

- **Endereço absoluto:**
 - somar (16384 + location) para o addr

Editor de *bitmaps*

Bitmap

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1																
2																
3																
4																
5																
6																
7																
8																
9																
10																
11																
12																
13																
14																
15																
16																

Function Type:

function ↕

Function Name:

draw

Generate Code >>

Generated Jack Code

```
function void draw(int location) {  
    let memAddress = 16384+location;  
    do Memory.poke(memAddress+0, 4064);  
    do Memory.poke(memAddress+32, 6192);  
    do Memory.poke(memAddress+64, 4752);  
    do Memory.poke(memAddress+96, 4112);  
    do Memory.poke(memAddress+128, 2336);  
    do Memory.poke(memAddress+160, 6192);  
    do Memory.poke(memAddress+192, 8200);  
    do Memory.poke(memAddress+224, 16644);  
    do Memory.poke(memAddress+256, 18724);  
    do Memory.poke(memAddress+288, 21396);  
    do Memory.poke(memAddress+320, 12312);  
    do Memory.poke(memAddress+352, 4112);  
    do Memory.poke(memAddress+384, 4368);  
    do Memory.poke(memAddress+416, 4752);  
    do Memory.poke(memAddress+448, 16120);  
    do Memory.poke(memAddress+480, 32508);  
    return;  
}
```

- Desenvolvido por Golan Parashi
- Disponível em: nand2tetris/projects/09

Boas práticas

- Para gráfico simples, utilize as funções do SO
- Para gráficos com alto desempenho, utilize suas próprias funções
- Se você precisa de *sprites* de alto desempenho utilize o editor de *bitmaps*
 - Projete o *sprite*
 - Gere o código Jack
 - Integre o código Jack gerado ao seu próprio código



Perspectiva

- **Jack é linguagem simples porém muito boa:**
 - fornece a maior parte dos elementos essenciais da programação procedural e orientada à objetos
- **Limitações:**
 - poucas estruturas de controle
 - algumas particularidades de sintaxe
 - não há herança
- **Tipos de dados:**
 - sistema de tipos primitivos
 - fracamente tipada

Motivação: uma linguagem simples que pode ser implementada através de um compilador simples.

Linguagem de alto nível: como será abordado?

- Programação em alto nível
 - Hello World
 - Programação procedural
 - Programação orientada à objetos
 - Processando Listas
- Especificação da linguagem Jack
 - Sintaxe
 - Tipos de dados
 - Classes
 - Métodos
- Desenvolvimento de aplicações
 - Aplicações Jack
 - Utilizando o Sistema Operacional
 - Exemplo de aplicação
 - Otimização de gráficos

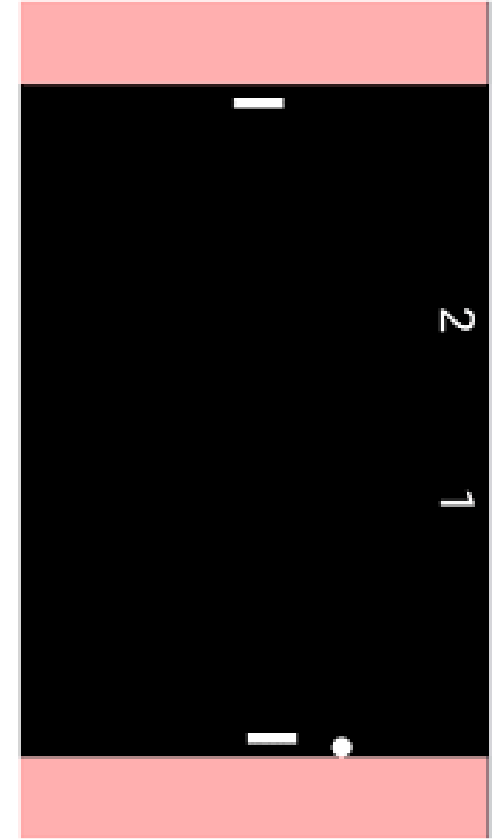
Especificação do Projeto

Pong para dois jogadores

Base: Projeto Pong localizado em `nand2tetris/projects/11/Pong`

Objetivo:

Estender o jogo original **Pong** para suportar dois jogadores humanos, um controlando a barra superior e o outro a barra inferior. O objetivo é fazer com que os jogadores rebatam a bola continuamente. O jogador que deixar a bola ultrapassar sua barra perde a partida, e uma mensagem de "Game Over" deve ser exibida na tela.



Especificação do Projeto

Requisitos Funcionais

Dois Jogadores Humanos:

Jogador 1: controla a barra **inferior**.

Controles: Teclas **esquerda/direita** (left, right)

Jogador 2: controla a barra **superior**.

Controles: Teclas alternativas, como a (esquerda) e d (direita), ou qualquer par disponível que permita controle simultâneo.

Barra Superior Adicional:

Implementar uma segunda barra, espelhada na parte superior da tela.

Ambas as barras devem ter limites de movimentação e comportamento idênticos.



UFRPE

Especificação do Projeto

Requisitos Funcionais

3. Mecânica de Rebote:

A bola deve colidir corretamente com ambas as barras, mudando de direção ao ser rebatida.

4. Condições de Derrota:

Se a bola ultrapassar a barra **superior**, o **Jogador 2** perde.

Se a bola ultrapassar a barra **inferior**, o **Jogador 1** perde.



UFRPE

Especificação do Projeto

Requisitos Funcionais

5. Tela de Game Over:

- Quando um jogador perde, o jogo deve parar e exibir uma mensagem clara no centro da tela, como:
 - "Game Over - Jogador 1 perdeu" ou "Game Over - Jogador 2 perdeu"
- A mensagem pode ser exibida usando a API `Screen.drawString(...)`.

6. Reinício:

- Não é necessário implementar reinício automático. O jogo pode ser reiniciado manualmente ao recarregar o projeto.



Especificação do Projeto

- A implementação deve ser feita em Jack, a linguagem de alto nível usada no curso.
- O projeto deve manter a estrutura modular, com arquivos .jack separados para:
 - Main.jack
 - PongGame.jack
 - Ball.jack
 - Bat.jack (reutilizável para ambas as barras)
- É permitido reutilizar o código original do *****Pong*****, desde que comentado e adaptado de forma adequada.



Especificação do Projeto

Critérios de Avaliação

- Funcionalidade
- Legibilidade
- Domínio



UFRPE

Especificação do Projeto

Critérios de Avaliação

Funcionalidade: Implementação correta da barra superior	1,0 pt
Funcionalidade: Controle independente das duas barras	1,0 pt
Funcionalidade: Lógica de colisão e rebote funcional	1,0 pt
Funcionalidade: Implementação das regras de derrota	1,0 pt
Funcionalidade: Mensagem de 'Game Over' com o jogador perdedor	1,0 pt
Legibilidade: Organização e clareza do código (comentários, nomes, modularidade)	1,0 pt
Domínio do projeto: Conhecimento sobre o próprio código	4,0 pt

Especificação do Projeto

Entrega

- A solução deverá ser apresentada em sala de aula com duração de no máximo 10 minutos.
- Na apresentação faça uma explicação breve da estrutura do código, comentando:
 - As principais classes criadas;
 - A estratégia adotada para movimentação;
 - Outros detalhes relevantes da implementação.
- O Professor poderá fazer perguntas. Então estude bem o seu código.



Especificação do Projeto

Entrega

- Submeta também no classroom um arquivo .zip contendo todos os arquivos .jack modificados/criados.
- A submissão deve ser feita no classroom na atividade: Entrega do Projeto da Unidade 6



UFRPE



UNIVERSIDADE
FEDERAL RURAL
DE PERNAMBUCO

Capítulo 9 - Linguagem de Alto Nível

Elementos de Sistemas Computacionais

Professor: Lucas Cambuim
lucas.cambuim@ufrpe.br